

IE University

School of Science & Technology



The Impact of Post-Training Techniques on Feature Representations

Alberto Puliga

Bachelor of Computer Science & Artificial Intelligence

Supervised by:

Prof. E. Castelló Ferrer

Connection Science, MIT, Cambridge, USA

Robotics & AI Lab, School of Science & Technology, IE University, Spain

Abstract

Large language models reach their users not as raw next-token predictors but as the products of an extensive post-training pipeline—instruction tuning, reward modelling, and preference optimisation—that installs the characteristic behaviours of a deployed assistant on top of a base model trained purely on web text. Yet mechanistic interpretability has concentrated overwhelmingly on base models, leaving the internal effects of post-training comparatively unexamined at the representational level. This thesis addresses that gap by asking a specific, empirically answerable version of a contested question in the alignment literature: at the feature level, does instruction tuning introduce new features into a language model’s internal representations, or does it predominantly reorganise existing ones?

The question is operationalised using matched Sparse Autoencoders from the recent Gemma Scope 2 release, which for the first time makes publicly available directly comparable dictionaries for the pre-trained and instruction-tuned variants of a production-class language model (Gemma 3 1B). A four-stage pipeline is developed that collects residual-stream activations from both checkpoints over 5,000 FineWeb sequences, classifies features into preserved, modified, and variant-exclusive categories using decoder cosine similarity and activation correlation as complementary correspondence measures, attaches descriptive labels to a frequency-weighted sample of features through dual-LLM consensus annotation with inter-annotator reliability analysis, and cross-references the resulting classification against targeted differential activation tests on three behavioural probe datasets covering safety, instruction-following, and truthfulness.

The findings converge on a consistent descriptive picture. Cleanly preserved features are a small minority of the active feature set, and the features most behaviourally relevant to the characteristic prompts of an instruction-tuned model—instruction-shaped inputs and harmful content—are predominantly features already present and already selective in the base model, with instruction tuning amplifying and reweighting rather than introducing them. These findings provide the first feature-level evidence from a real instruction-tuned language model consistent with the “wrapper hypothesis” of Jain et al. (2024), previously tested only in synthetic settings, and motivate a reading of instruction tuning as a reorganisation of a pre-existing representational substrate rather than an installation of new computational structure—a framing with direct implications for how alignment teams should

reason about the base-model origins of deployed safety behaviours.

1 Introduction

1.1 Problem and Motivation

Contemporary large language models reach the user not as the products of pre-training alone but as the products of an extensive post-training pipeline that layers supervised instruction tuning, preference learning from human feedback, constitutional AI, and direct preference optimisation on top of a base model that has merely learned to predict next tokens on web-scale text [2, 6, 23, 29, 39]. It is this post-training stack, not the base model, that gives the deployed system its characteristic behaviours: following instructions, answering questions in a particular register, refusing harmful requests, and staying in the role of an assistant. Yet most mechanistic interpretability research on large language models has concentrated on base models—from Olah et al. [22]’s Zoom In essay on circuits to the indirect object identification circuit of Wang et al. [34], the factual association edits of Meng et al. [19], and the feed-forward key–value memories of Geva et al. [11]—leaving the internal mechanics of post-training comparatively under-studied. The result is a gap between the layer of the language model that interpretability understands best and the layer of the language model that users actually interact with.

Closing this gap is not a matter of scientific completeness alone; it has concrete implications for alignment practice. If we do not understand at a representational level what instruction tuning does to a model, we cannot know whether an observed safety behaviour in a deployed model is a robust property of the model’s computation or a brittle veneer that can be undone by further fine-tuning, jailbroken by adversarial prompting, or silently circumvented by deceptive alignment [12, 40]. The current answer to these questions is largely behavioural: we probe the deployed model, observe whether it refuses harmful prompts, and draw conclusions. This is informative but it is an outside-the-box view. What we lack is an inside-the-box account of which internal representations the post-training stack builds, reuses, or rearranges, and which of them carry the weight of the behaviours we care about.

The mechanism of instruction tuning in particular is the subject of a specific, contested, and empirically answerable debate. One widely cited position is the superficial alignment hy-

pothesis associated with Zhou et al. [38]: the observation that a surprisingly small amount of high-quality instruction-following data is sufficient to produce strong instruction-following behaviour in a large base model, implying that instruction tuning may not add new representational content so much as teach the model to expose existing content in a task-appropriate register. A competing position, less often articulated as a named hypothesis but implicit in much of the alignment literature, is that instruction tuning installs new computational structure—new features, new circuits, or new decision procedures—that did not previously exist in the base model and that are necessary for the characteristic behaviours of the instruction-tuned variant. These two positions predict observably different things about the internal representations of the instruction-tuned model, and whether a specific feature in the instruction-tuned model’s layer 13 residual stream has a counterpart in the base model’s layer 13 residual stream is exactly the kind of observation that can distinguish them.

Until recently, testing this distinction at the representational level was not practical. The tools available to mechanistic interpretability—probing classifiers, activation patching, attention pattern inspection, residual stream decomposition—either measured aggregate behavioural properties of the model or studied individual circuits one at a time, and neither approach gave a vocabulary in which two models’ entire representational states could be systematically compared. The arrival of Sparse Autoencoders as a practical interpretability tool, beginning with the toy-model superposition work of Elhage et al. [9] and early monosemanticity results [7], and maturing through the Gemma Scope release and its successors [17], changes this picture. SAEs give a feature-level dictionary of a model’s residual-stream activations, with each dictionary entry corresponding to a direction in activation space that tends to activate on semantically legible patterns. Two models in the same architectural family, or—crucially for the present work—two checkpoints of the same model before and after post-training, can now be equipped with SAEs trained on the same residual-stream location, and their feature dictionaries can be compared directly. A feature that exists in both dictionaries, points in the same direction, and activates on the same inputs is a feature that has been preserved across post-training. A feature that exists in only one of the two dictionaries, or whose direction or functional behaviour has shifted, is a feature that has been modified, added, or removed. This comparison is the specific operational-

isation of the debate between the superficial-alignment reading and the new-capability reading of instruction tuning, and it is what the present thesis undertakes.

The motivation for studying instruction tuning specifically, rather than the full post-training stack, is both practical and scientific. Practically, instruction tuning is the first and simplest stage of post-training, and its effects can be isolated from those of reinforcement-learning-based stages provided that matched Sparse Autoencoders exist for both the base model and the supervised-fine-tuned checkpoint. The Gemma Scope 2 release is, at the time of writing, the only such matched-SAE release for any production-class language model, and the Gemma 3 1B base and instruction-tuned checkpoints are its only covered pair—a constraint that determines the model selection of this thesis and is discussed in detail in Subsection 2.1. Scientifically, instruction tuning is also the stage of post-training whose mechanism is most directly addressed by the superficial-alignment literature and by the recent line of mechanistic fine-tuning work [8, 13, 27], which means that a feature-level study of instruction tuning connects immediately to a live debate in the interpretability and alignment communities rather than starting one from scratch.

The research question that follows from this motivation is stated in its final form in Subsection 1.2 after the literature review, but can be previewed here: at the feature level, does instruction tuning introduce new features into a language model’s internal representations, or does it reorganise existing ones? The rest of this thesis sets out to answer that question for one specific model pair, using one specific interpretability tool, at one specific layer, with three behaviourally grounded probe datasets, and with explicit attention to the robustness of the answer across reasonable perturbations of its free parameters. What the thesis finds, in anticipation of Section 4, is a descriptive picture consistent with the reorganisation reading: the behaviourally relevant features of the instruction-tuned model are predominantly reshaped versions of features already present in the base model, not new features introduced from scratch. The conclusion is narrower than the full debate demands—it speaks to one layer of one model and is correlational rather than causal—but it is a concrete, reproducible data point on a question that has so far been addressed mostly at the behavioural and circuit level, and it provides a methodology that can be reused as matched Sparse Autoencoders become available for further model pairs.

1.2 Prior Art and Research Question

This subsection reviews the prior work that frames the research question of this thesis. It is organised thematically rather than chronologically, in four subsections. Subsubsection 1.2.1 covers the representational foundations on which this work rests: the linear representation hypothesis, superposition, and dictionary learning as responses to the problem that individual neurons are not the right unit of analysis for understanding what a language model has learned. Subsubsection 1.2.2 turns to Sparse Autoencoders as a concrete interpretability tool, tracing the line from early toy models through to the Gemma Scope releases that make the present work possible. Subsubsection 1.2.3 surveys what interpretability currently knows about the effect of post-training on internal representations, dividing the landscape into the superficial-alignment line of argument, the mechanistic fine-tuning line of work, and the representation-engineering and activation-steering work that sits adjacent to both. Subsubsection 1.2.4 identifies the specific gap in this literature that the present thesis addresses and states the research question in its final form.

1.2.1 Representational Foundations: Superposition, Linear Features, and the Case for Dictionary Learning

The unit-of-analysis problem in mechanistic interpretability is older than interpretability as a field: it was already implicit in early work on distributed word representations [20], which showed that directions in an embedding space could carry semantically meaningful structure that could not be read off any individual coordinate. Individual neurons, or individual coordinates, are generally not the right unit of analysis because the same neuron participates in many unrelated computations. Radford, Jozefowicz, and Sutskever [28] observed this in the context of character-level language models, where a single “sentiment neuron” was identified but turned out to be an exception rather than the rule. Olah, Mordvintsev, and Schubert [21] and Olah et al. [22] articulated the more general view that the right unit of analysis is the feature—a direction in activation space that corresponds to a semantically legible pattern—and that features need not correspond to individual neurons.

The formal explanation for why individual neurons often fail as a unit of analysis is the phe-

nomenon of *superposition*, characterised in depth by Elhage et al. [9]. The argument proceeds from a simple observation: a neural network with n neurons cannot in general represent more than n orthogonal directions exactly, but it can represent many more than n near-orthogonal directions approximately, provided the represented features are sparse, that is, inactive on most inputs. A sparse set of features can be packed into a lower-dimensional space with interference that is tolerable in expectation, and the resulting representation assigns each feature a direction that is in general a linear combination of several neurons. The consequence is that single-neuron interpretation is not merely aesthetically limited but methodologically incorrect for sufficiently overcomplete learned representations: the neurons of such a network will be polysemantic by design. Scherlis et al. [31] extend this analysis to a more rigorous account of polysemanticity as a function of capacity, and Park, Choe, and Veitch [24] formalise the linear representation hypothesis—that meaningful features in language model activations correspond to directions rather than to nonlinear manifolds—as a testable structural claim about transformer internals.

Superposition motivates dictionary learning as a natural remedy. If the true representational units of the network are features that live as sparse linear combinations of neurons, then the correct way to recover them is to learn a dictionary of feature directions and an encoder that assigns each activation a sparse code over that dictionary. This is a classical idea in signal processing, and its application to neural network interpretability received its first major language-model demonstration in Cunningham et al. [7], who showed that sparsely activated dictionaries trained on transformer activations recovered features substantially more interpretable than individual neurons on the same models. Concurrent work in Anthropic’s Towards Monosemanticity line [4] scaled the approach to larger transformers and established much of the vocabulary—“features,” “monosemantic directions,” and “activating examples”—that has become standard in the SAE literature. The methodological premise of the present thesis rests on this line of work: that features recovered by dictionary learning on a transformer’s residual stream are a meaningful representational unit, and that two transformers whose residual streams are dictionary-decoded in the same way can be compared at the level of their features.

1.2.2 Sparse Autoencoders as an Interpretability Tool

Sparse Autoencoders (SAEs) are the specific form of dictionary learning used in this thesis and in most recent SAE-based interpretability work. An SAE takes an activation $h \in \mathbb{R}^D$ from some location in a transformer, typically the residual stream at a specified layer, encodes it into a sparse non-negative feature vector $f(h) \in \mathbb{R}^F$ with $F > D$, and decodes it through a learned dictionary $W_{\text{dec}} \in \mathbb{R}^{F \times D}$ to produce a reconstruction $\hat{h} = W_{\text{dec}}^\top f(h)$. Training minimises reconstruction loss together with an explicit or implicit sparsity penalty, so that only a small subset of the F features activate on any given input. The resulting decoder rows are the feature directions, and the encoder gives the sparse code.

Early SAE work used the classical formulation with an L_1 sparsity penalty on the encoder output, but this formulation has known failure modes. L_1 penalties shrink active features toward zero, biasing both reconstruction and downstream analysis, and the fixed sparsity regime is difficult to tune across layers and scales. Two important methodological refinements emerged in response. Rajamanoharan et al. [30] introduce gated SAEs, which decouple the decision of whether a feature is active from the magnitude of its activation, reducing the shrinkage bias of the L_1 formulation, and later work in the same line introduces the JumpReLU encoder used in Gemma Scope, which makes the feature activation threshold explicit and trainable. Gao et al. [10] provide a systematic study of how SAE quality scales with dictionary width, training compute, and sparsity target on large language models, and establish empirical scaling laws that guided the hyperparameter choices of subsequent SAE releases. Karvonen et al. [15] propose a comprehensive evaluation benchmark, SAEBench, that measures SAE quality along multiple axes beyond reconstruction loss, including feature interpretability, probe-based downstream utility, and robustness to perturbation.

The most important concrete artefact for the present thesis is the Gemma Scope release [17]. Gemma Scope made publicly available a large family of trained Sparse Autoencoders across every layer and several widths of Gemma-family models, and Gemma Scope 2 extended this to include matched releases for the Gemma 3 1B base and instruction-tuned checkpoints used throughout this thesis. The scale and reproducibility of Gemma Scope changed what was practical to ask of SAE-based interpretability: researchers could now write papers that assumed

the existence of high-quality trained SAEs for a production-class language model rather than training their own from scratch. Peng et al. [26] further argues that SAEs are most useful for discovering unknown concepts rather than acting on previously known ones, a caution that is relevant to the targeted analysis stage of the present thesis, which uses behavioural probe datasets to let the feature space surface its own concepts rather than prescribing them in advance.

1.2.3 Post-Training and Its Effects on Internal Representations

Post-training is the umbrella term for everything that happens to a language model after pre-training on next-token prediction. The standard modern pipeline begins with supervised instruction tuning [35, 38], continues with reward modelling [6, 39], and often concludes with reinforcement learning from human feedback [23]. Constitutional AI [2] and related variants replace or augment the human feedback with model-generated feedback grounded in an explicit set of principles, and Rafailov et al. [29]’s Direct Preference Optimisation reformulates the RL stage as a direct optimisation problem that sidesteps the reward model altogether. The broader helpful-and-harmless line of Bai et al. [3] frames the goal of this pipeline as producing a model that is simultaneously useful to users and safe to deploy.

The question of what all this does to the internal representations of the model has been addressed from three distinct angles in the interpretability literature, and the research question of this thesis sits at the intersection of all three.

The superficial alignment line of argument. The strongest statement of this view is Zhou et al. [38]’s LIMA paper, whose central empirical finding is that 1,000 carefully curated instruction-response examples are sufficient to produce strong instruction-following behaviour in a large base model, comparable to models fine-tuned on orders of magnitude more data. The authors’ interpretation is explicit: a model’s knowledge and capabilities are learned almost entirely during pre-training, while alignment teaches it which subdistribution of formats should be used when interacting with users. This is a behavioural-evidence argument and does not itself commit to any particular representational mechanism, but it makes a strong prediction about what an interpretability study would find: that the substrate for instruction-following behaviour

is already present in the base model and that fine-tuning primarily affects which features are read out and in what register, rather than which features exist. The LIMA paper does not test this prediction at the representational level, but it frames the exact empirical question that the present thesis answers.

The mechanistic fine-tuning line. A parallel line of work approaches the same question from a mechanistic rather than behavioural angle, using circuit-level and probe-level tools to ask what structures inside the model change when it is fine-tuned. The foundational paper for the present thesis is Jain et al. [13], which studies fine-tuning in controlled synthetic settings using compiled Tracr transformers and probabilistic context-free grammars, and finds that fine-tuning rarely alters underlying model capabilities, instead learning a minimal transformation, or “wrapper,” on top of them. The wrapper hypothesis—that fine-tuning installs a minimal re-routing of existing capabilities rather than building new ones—is the specific claim this thesis tests in a new setting. Jain et al.’s evidence comes from pruning, probing, and reverse fine-tuning in synthetic environments where ground truth is known; it does not come from SAE-based feature-level analysis and does not apply directly to a production-class language model. Extending the wrapper hypothesis to a real instruction-tuned LLM at the feature level is the specific extension attempted here.

Adjacent to Jain et al. is Prakash et al. [27], which uses circuit-level tools in a more realistic setting to show that fine-tuning for entity tracking does not build a new circuit but rather enhances the selectivity and magnitude of a circuit already present in the base model. Du et al. [8] extends this picture further and is directly concerned with which aspects of model behaviour are most affected by post-training at the representational level. Xu et al. [37] works at a different temporal resolution, within pre-training itself, but contributes the methodological observation that feature-level analysis can track representational change over training, which is the temporal analogue of the checkpoint-comparison method used in this thesis.

Representation engineering, activation steering, and the causal side. A third adjacent literature uses representations not only as objects of interpretation but as targets of intervention. Zou et al. [41] propose a top-down approach in which high-level concept directions are extracted from model activations using contrast pairs and then used to monitor, probe, and

steer model behaviour, demonstrating that properties such as honesty, refusal, and emotional valence can be read from and written to the residual stream via linear operations. This line of work provides the causal counterpart to correlational feature-level analyses: where SAE-based work asks what features exist in a model, representation engineering asks what happens when those features are perturbed. The present thesis makes a limited excursion into this territory via the steering analysis reported in Subsection 5.1.3.

The jailbreak and adversarial-attack literature [12, 40] provides the motivating edge for this line of work. If safety behaviours are a thin wrapper over a base-model capability substrate, then adversarial prompts that bypass the wrapper should be able to surface capabilities that the instruction-tuned model would otherwise not exhibit. A feature-level picture of which representations are shared between base and instruction-tuned checkpoints, which is what this thesis provides, is directly relevant to understanding where this surfacing can and cannot reach.

Similarity-of-representations work. One further piece of adjacent literature deserves mention: Kornblith et al. [16]’s work on Centered Kernel Alignment (CKA) and related similarity metrics for comparing neural network representations. CKA and its variants give an aggregate measure of how similar two networks’ representations are at a given layer, and have been used extensively to compare fine-tuned models to their base models. Aggregate similarity measures are complementary to the feature-level comparison of this thesis: CKA tells you how much two representations differ overall, whereas a feature-level analysis tells you which specific axes differ and how. The present thesis does not compute CKA between the PT and IT residual streams, but the feature-level results reported later could in principle be sanity-checked against such a computation in future replications.

1.2.4 The Gap and the Research Question

The literature reviewed in Subsubsections 1.2.1–1.2.3 establishes three things. First, Sparse Autoencoders are now a sufficiently mature tool to give a feature-level vocabulary for the residual stream of a production-class language model, and matched SAEs exist for the Gemma 3 1B base and instruction-tuned checkpoints via Gemma Scope 2. Second, there is a live and contested hypothesis that instruction tuning primarily re-exposes existing base-model capabilities

rather than installing new ones, most prominently in the LIMA paper but implicit across much of the alignment literature. Third, this hypothesis has already been tested with non-SAE tools in synthetic settings and with circuit-level tools in specific task settings, and the results of both lines of work are broadly consistent with it in their respective domains, but it has not been tested at the feature level in a real instruction-tuned LLM using matched Sparse Autoencoders.

The specific gap this thesis addresses is the intersection of these observations: there is no existing work that uses matched Sparse Autoencoders to compare the feature-level representations of a real pre-trained and instruction-tuned language model at the same layer and to test, descriptively, whether the behaviourally relevant features of the instruction-tuned model are preserved, modified, or newly introduced relative to the base model. The Gemma Scope 2 release makes this comparison possible for the first time on Gemma 3 1B, and the absence of prior work in this intersection is what motivates the pipeline described in Section 2.

The research question that follows is: at the feature level, does instruction tuning introduce new features into a language model’s internal representations, or does it predominantly reorganise existing ones? Specifically, when the residual stream of a pre-trained and an instruction-tuned variant of Gemma 3 1B is decoded with matched Sparse Autoencoders at layer 13, are the features most behaviourally relevant to instruction-following and safety-related prompts better characterised as preserved, modified, or exclusive to one of the two checkpoints?

The question is deliberately narrower than the full superficial-alignment versus new-capability debate. It restricts itself to a single layer, a single model pair, and a single post-training stage. It also restricts itself primarily to a descriptive and correlational framing, even though the thesis includes a limited causal steering check, for reasons of scope discussed in Section 2 and Subsection 5.1. The narrowness is intentional: a question that can be answered precisely with the available tools is more valuable than a broader question that cannot. The answer to this narrower question, as later sections report and interpret, provides one concrete feature-level data point in a debate that has so far been conducted mostly at the behavioural and circuit level.

This question is operationalised through three sub-questions:

- RQ1: What proportion of features at layer 13 of Gemma 3 1B are preserved, modified, PT-exclusive, or IT-exclusive between the base and IT checkpoints, and how robust is

that proportion to the choice of matching thresholds?

- RQ2: When the IT model is probed with safety-relevant and instruction-following prompts, do the most differentially active features map onto IT-exclusive entries, as a strict-creation reading would predict, or onto features that the matching pipeline already paired with a PT counterpart?
- RQ3: Does causal manipulation via activation steering of the features identified in RQ2 produce the behavioural changes that their labels predict, in both the base and IT models?

1.3 Main Contribution

The main contribution of this thesis is a four-stage empirical pipeline for testing whether instruction tuning introduces new features into a language model’s internal representations or predominantly reorganises pre-existing ones, together with the first application of this pipeline to a real instruction-tuned language model using matched Sparse Autoencoders. Applied to the pre-trained and instruction-tuned variants of Gemma 3 1B at the residual stream of layer 13, using the `gemma-scope-2-1b-pt-res` and `gemma-scope-2-1b-it-res` Sparse Autoencoders from the Gemma Scope 2 release, the pipeline produces empirical evidence that the features most behaviourally relevant to instruction-following and safety-related prompts are predominantly reorganised pre-existing features rather than newly introduced ones—a descriptive result consistent with the wrapper hypothesis of Jain et al. [13], here tested for the first time in a real instruction-tuned LLM at the feature level.

The concrete deliverables of the thesis, each of which is elaborated in the chapters that follow, are as follows.

The first deliverable is the **four-stage pipeline itself**. Stage one collects residual-stream activations from 5,000 FineWeb sequences for both the pre-trained and the instruction-tuned checkpoints at layer 13 and encodes them with the matched Gemma Scope 2 Sparse Autoencoders, producing per-feature activation frequencies, mean activations, and per-sequence mean activation vectors for each model. Stage two computes two complementary feature-correspondence measures between the two checkpoints—decoder cosine similarity and acti-

variation correlation—and uses them jointly to partition the active feature sets of both checkpoints into four classes: *preserved* features (strong correspondence on both measures), *modified* features (strong correspondence on only one of the two measures), *PT-exclusive* features (active in the base model with no strong counterpart in the instruction-tuned dictionary), and *IT-exclusive* features (active in the instruction-tuned model with no strong counterpart in the base dictionary). The classification is subjected to a 25-point threshold sensitivity grid to verify that its qualitative shape is not an artefact of the primary operating point chosen for the interpretive stages. Stage three attaches a descriptive vocabulary to a frequency-weighted sample of 50 features from each class by submitting their top-activating contexts to two independent large language model annotators (Claude Haiku 4.5 and GPT-5.4-nano) under identical prompts and identical structured-output schemas, and constructs a consensus label by confidence-weighted tiebreak on disagreements, treating the inter-annotator agreement rate and the structure of disagreement cases as first-class reliability diagnostics rather than as noise to be filtered away. Stage four runs a targeted differential activation analysis on three external behavioural probe datasets—JailbreakBench for safety-relevant prompts, Stanford Alpaca for instruction-following prompts, and TruthfulQA for a truthfulness contrast—using Welch’s two-sample t -tests with an explicit $p < 0.01$ threshold and a minimum conditional-mean activation filter, and cross-references the resulting ranked lists of differential features against the four-way classification of stage two.

The second deliverable is the empirical findings that the pipeline produces. These are reported in full in Chapter 6 and summarised in Chapter 8, and consist of three convergent strands of descriptive evidence. The classification of stage two finds that cleanly preserved features are a small minority of active features at the primary operating point, that the modified class is dominant across the entire threshold sensitivity grid, and that both checkpoint-exclusive classes are sizeable but—as the targeted analysis of stage four shows—not behaviourally central. The dual-LLM consensus labelling of stage three finds that inter-annotator agreement is moderate overall and is concentrated on the preserved and modified classes, that disagreement concentrates at specific identifiable taxonomic boundaries (most prominently the boundary between low-level linguistic features and instruction-style features), that preserved features are almost entirely

low-level linguistic and syntactic in character, and that instruction-style features are essentially absent from the preserved class and concentrated in the modified class instead. The targeted differential analysis of stage four finds that the top-ranked features responding to instruction-shaped prompts in the instruction-tuned model are overwhelmingly labelled as instruction-style and that several of them—most saliently, specific feature indices that can be identified by number in Chapter 6—appear in the corresponding top-ranked lists for the base model on the same prompts with large effect sizes, meaning that these features are already present and already selective in the base model. The same cross-checkpoint overlap pattern holds, more weakly, for features responding to harmful prompts under the JailbreakBench contrast.

The third deliverable is the methodological template that the pipeline represents. The four stages are loosely coupled and can be reused independently: the feature-matching step of stage two can be applied to any pair of matched SAEs; the dual-LLM consensus labelling of stage three can be applied to any set of SAE features with top-activating contexts; the targeted differential analysis of stage four can be applied to any set of features identified by any other means. The template is the first published application of matched Sparse Autoencoder comparison between a base and an instruction-tuned checkpoint of a production-class language model, and it defines a reusable protocol for any future study that will have access to matched SAEs for a different model pair.

What the thesis does not claim is equally important and equally explicit. The descriptive evidence supports a correlational reading, not a causal one. Chapter 7 is explicit about the absence of a behaviourally grounded causal validation experiment, reports the partial results of a cross-SAE direction robustness check as a structural observation rather than as a causal finding, and names the absence of full causal validation as the primary limitation of the work and the most important piece of future work. Chapter 8’s verdict on the wrapper hypothesis is therefore a calibrated “consistent with” rather than a confirmatory “supports,” and the strongest version of the hypothesis—that instruction tuning creates no new features at all—is explicitly rejected as unsupported by the evidence. What the thesis does claim, and what the pipeline and findings support, is the weaker and more defensible position that at the residual stream of layer 13 in Gemma 3 1B, the features carrying the behavioural signatures of instruction tuning

are predominantly reorganised from a pre-existing base-model feature population rather than introduced from scratch. This is the contribution, and the rest of the thesis is the defence of it.

2 Methodology

This chapter describes the empirical pipeline used to test whether instruction tuning introduces new features into a language model’s internal representations or predominantly reorganises pre-existing ones. The pipeline is organised as four stages that map directly onto the research question formulated in §3.4. Stage one (§5.1–5.3) fixes the model pair, the interpretability tool, and the activation corpus, ensuring that the pre-trained and instruction-tuned variants are compared in a shared geometric space using matched Sparse Autoencoders. Stage two (§5.4–5.5) defines a feature-level correspondence between the two checkpoints based on two complementary measures—decoder direction similarity and activation correlation—and uses a grid over classification thresholds to check that the resulting four-way partition (preserved, modified, PT-exclusive, IT-exclusive) is not an artefact of any single threshold choice. Stage three (§5.6) attaches a descriptive vocabulary to the sampled features through automated dual-LLM labelling with consensus construction, providing both the interpretive layer needed to discuss what individual features mean and a form of inter-annotator reliability check. Stage four (§5.7) grounds the classification in behaviourally meaningful prompts by computing targeted differential activations across three contrast datasets—JailbreakBench, Stanford Alpaca, and TruthfulQA—using Welch’s t -tests to identify features whose activation is selectively elevated on safety, instruction-following, or truthfulness probes. All code was implemented in a single Jupyter notebook using the SAELens and HuggingFace Transformers libraries; the full implementation and reproducibility artefacts are discussed in Chapter 6 alongside the results.

2.1 Model Pair and Sparse Autoencoders

The experimental subject of this thesis is the Gemma 3 1B model family released by Google, specifically the pre-trained base checkpoint `google/gemma-3-1b-pt` and its instruction-tuned counterpart `google/gemma-3-1b-it`. Both checkpoints share the same architecture—26 transformer layers with a hidden size of 1152—and were loaded at `bfloat16` precision using the HuggingFace `transformers` library. The Gemma 3 family was chosen over larger alternatives for two complementary reasons. First, the 1B scale is small enough to fit entire pre-trained

and instruction-tuned variants, their matched Sparse Autoencoders, and the activation buffers for a 5,000-sequence corpus into a single commodity GPU, which was a hard constraint for reproducibility and iteration speed. Second, and more importantly, this is the smallest scale at which publicly released Sparse Autoencoders exist for both a base and an instruction-tuned checkpoint of the same architecture, via the Gemma Scope 2 release [17]. This matching of SAEs to both variants is essential for the research question and is discussed further in §5.2.

Sparse Autoencoders are the primary interpretability tool used throughout this thesis. Following the dictionary-learning framing of [4] and the formal setup of Cunningham et al. [7], a Sparse Autoencoder learns an overcomplete dictionary of feature directions $\{d_i\}_{i=1}^F$ in the residual stream space of a target transformer layer, together with a sparse encoder that assigns each activation $h \in \mathbb{R}^D$ a non-negative feature vector $f(h) \in \mathbb{R}^F$ such that the reconstruction $\hat{h} = \sum_i f_i(h) d_i$ approximates h in the mean-squared sense. The resulting feature directions are intended to correspond to semantically interpretable axes of variation, mitigating the superposition problem described by Elhage et al. [9] whereby individual neurons participate in many unrelated computations. The specific SAEs used in this work come from the Gemma Scope 2 release: `gemma-scope-2-1b-pt-res` for the base model and `gemma-scope-2-1b-it-res` for the instruction-tuned model, both at the `layer_13_width_16k_10_medium` configuration. Both SAEs have a dictionary width of 16,384 features, use the JumpReLU encoder of Rajamanoharan et al. [30], and were trained by Google DeepMind at a comparable expected L_0 sparsity, making them directly comparable as feature dictionaries for the same underlying architecture.

Layer 13 was chosen as the target layer because it sits at the midpoint of the 26-layer stack, which is where prior work on Gemma Scope and on residual-stream SAEs more broadly has reported the richest mixture of syntactic and semantic features. Middle layers are generally understood to be where the abstract features relevant to downstream behaviour are most legible, whereas earlier layers are dominated by token-level and low-level syntactic features and later layers are increasingly output-oriented. A single-layer choice is a scoping decision rather than a methodological claim, and its implications for generalisation are discussed explicitly in Chapter 7.

2.2 Why Gemma 3 1B and not Gemma 2 2B

An earlier version of this project planned to use the Gemma 2 2B pair, since at the time Gemma 2 was the default target of the original Gemma Scope release and of most downstream interpretability work. The migration to Gemma 3 1B was forced by a single but decisive consideration: at the time this project was scoped, the original Gemma Scope release provided publicly released SAEs only for the base `gemma-2-2b-pt` checkpoint, and no matched SAE existed for the instruction-tuned `gemma-2-2b-it` variant. The research question of this thesis—whether the same feature space is preserved, modified, or newly populated across checkpoints—is not answerable without SAEs trained on both variants, because there is no meaningful way to compare the feature dictionary of one checkpoint to a dictionary that does not exist for the other. Training an IT SAE ourselves was considered but ruled out on reproducibility and scope grounds: doing so would have introduced a large number of free hyperparameters whose choices would be impossible to validate against any reference, and the resulting dictionary would not be a recognised community artefact that other researchers could reuse. The Gemma Scope 2 release [17] resolved this problem by providing matched SAEs for both `gemma-3-1b-pt` and `gemma-3-1b-it` at identical layer and width configurations, and the project was migrated accordingly. One additional implementation detail is worth recording: Gemma 3 1B requires `bfloat16` rather than `float16` for inference, as the narrower dynamic range of `float16` produced overflow-driven NaN activations during early testing and would have silently corrupted the activation buffer. All model forward passes in this project therefore use `bfloat16`.

2.3 Activation Collection

The activation corpus consists of 5,000 sequences drawn from the HuggingFace `HuggingFaceFW/fineweb` dataset, specifically the `sample-10BT` split, accessed in streaming mode with a fixed random seed for reproducibility. Each sequence was tokenised with the shared Gemma tokenizer and truncated to exactly 256 tokens; sequences shorter than 256 tokens after tokenisation were discarded rather than padded, which guarantees that every retained sequence contributes the same

number of activation rows and avoids the need to mask padding tokens during feature statistics. The 5,000-sequence figure was a deliberate downscaling from an initial 50,000-sequence plan, chosen after a pilot confirmed that the SAE feature statistics most important to the pipeline—per-feature activation frequency, mean activation, and per-sequence mean activation vectors used in the activation-correlation step of §5.4—stabilised well below 50,000 sequences and that the additional data primarily increased runtime without changing the qualitative shape of the results. The final corpus contains approximately 1.275 million activation rows per model (5,000 sequences $\times$ 255 tokens after the first token of each sequence is dropped, as described below).

For each of the two models, activations were collected from the residual stream output of layer 13 using a PyTorch forward hook registered on the corresponding `model.model.layers[13]` module. Sequences were processed in batches of four, and within each batch the first token of each sequence was dropped before SAE encoding, because the first residual-stream position of a causal transformer carries no context from other tokens and its activation statistics are atypical. The resulting activation matrix was moved to CPU memory in `float32` and then re-encoded through the SAE in chunks of 64 rows at a time; this chunked encoding was necessary because naively encoding the entire activation matrix in a single GPU call caused out-of-memory errors on the 16,384-wide feature dictionary, and the one-sequence-at-a-time alternative was too slow to be practical. For each chunk, the SAE encoder, decoder, and reconstruction residual were computed, and three running statistics were updated per feature: a sum of activations, a sum of squared activations, and a count of non-zero activations. These running statistics were used after all batches to compute the per-feature mean activation $\bar{f}_i$ and activation frequency ρ_i . In parallel, a per-sequence mean activation vector was computed for each sequence, producing a matrix of shape 5000×16384 that was later used as the input to the activation-correlation step of §5.4. Finally, two SAE quality metrics were tracked throughout encoding: the fraction of variance unexplained (FVU) of the reconstruction and the mean L_0 count, computed as the average number of non-zero features per activation row. These were reported at the end of each encoding pass as sanity checks that the pre-trained SAEs were operating within their expected regime on the FineWeb corpus rather than failing silently.

The end state of this phase is, for each of the two models, a complete summary of SAE feature behaviour over the corpus consisting of activation frequency, mean activation, and per-sequence mean activation vectors—in addition to the SAE decoder matrices themselves, which are loaded once and reused. These four objects are the only inputs required for the feature matching and classification stage described next.

2.4 Feature Matching and Four-Way Classification

The central methodological construct of this thesis is the four-way classification of SAE features into preserved, modified, PT-exclusive, and IT-exclusive categories. Two complementary similarity measures are used to build this classification: decoder direction similarity, which is a purely geometric quantity computed from the SAE weights alone, and activation correlation, which is a functional quantity computed from how features co-activate on the shared FineWeb corpus. The rationale for using two measures rather than one is that either alone is insufficient. Decoder similarity is cheap to compute and captures whether the two dictionaries point in the same directions in residual-stream space, but it ignores whether the features with similar directions are actually used similarly by the two models. Activation correlation captures functional equivalence—whether a feature in the base model and a feature in the instruction-tuned model light up on the same sequences—but depends on the choice of similarity measure for matching features in the first place, and is dominated by noise for features that are rarely active. Using both measures jointly, and requiring agreement between them for the strongest category (preserved), is intended to produce a classification that is robust to the weaknesses of either measure in isolation.

Decoder direction similarity is computed as follows. Let $D^{\text{PT}} \in \mathbb{R}^{F \times d}$ and $D^{\text{IT}} \in \mathbb{R}^{F \times d}$ denote the decoder matrices of the PT and IT SAEs respectively, where $F = 16,384$ is the dictionary width and d is the residual stream dimension. Both matrices are ℓ_2 -normalised row-wise so that every decoder direction is a unit vector, and the cosine similarity matrix is implicitly defined as $S = D^{\text{PT}}(D^{\text{IT}})^\top$. For each PT feature i , the best matching IT feature is found by taking $j^*(i) = \arg \max_j S_{ij}$ and its maximum cosine similarity is recorded as s_i^{max} . The same procedure is run in the reverse direction to find, for each IT feature, the best matching PT

feature. This two-directional matching matters because the best PT-to-IT match is not in general the same as the best IT-to-PT match, and the direction of matching affects which feature of each pair “owns” the comparison. In practice the computation is done in chunks of 1,024 rows to keep the intermediate similarity tensor within GPU memory limits, but this is a performance rather than methodological detail.

Activation correlation is computed over the per-sequence mean activation matrices collected in §5.3. For each PT feature i and its best-matched IT feature $j^*(i)$, the Pearson correlation of their per-sequence activation vectors is computed over the 5,000 sequences. A small numerical guard is applied to avoid division by zero on features whose per-sequence activation vector has zero variance (which occurs for features that either never activate or activate at exactly constant magnitude). The resulting scalar $\rho_i \in [-1, 1]$ measures whether the PT feature and its geometric nearest neighbour in IT space are functionally correlated on the same inputs. The same quantity is computed in the reverse direction for IT features.

With these two measures in hand, the four-way classification at thresholds (τ_d, τ_a) is defined as follows. A PT feature i is called *preserved* if both $s_i^{\max} \geq \tau_d$ and $\rho_i \geq \tau_a$, meaning that its best geometric match in IT space is both close in direction *and* functionally correlated with it on the shared corpus. It is called *modified* if exactly one of the two conditions holds—that is, if the direction is preserved but the function has drifted, or the function is preserved but the direction has rotated. It is called *PT-exclusive* if neither condition holds, meaning that no strong geometric or functional counterpart exists in the IT dictionary. The fourth category, *IT-exclusive*, is defined symmetrically on IT features using the reverse-direction best matches and their activation correlations, and identifies IT features with no strong counterpart in the PT dictionary. Together, the preserved, modified, and PT-exclusive classes partition the active PT features, while the IT-exclusive class covers the complementary set of active IT features; a feature is treated as *active* if its activation frequency exceeds 0.1% of tokens, a threshold intended to remove dead features from the analysis without being restrictive enough to bias the category distribution. The active-feature threshold is reported explicitly alongside the classification results in Chapter 6 so that readers can distinguish distributional findings from artefacts of feature pruning.

2.5 Threshold Selection and Sensitivity Analysis

The classification described in §5.4 depends on two free parameters, τ_d and τ_a , and any single choice of these parameters would leave the results vulnerable to the criticism that the reported proportions of preserved, modified, and exclusive features are an artefact of a lucky threshold. To address this, the full classification is computed on a grid of 25 threshold pairs, with $\tau_d \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$ and $\tau_a \in \{0.1, 0.2, 0.3, 0.5, 0.7\}$. For each pair, the number of preserved, modified, PT-exclusive, and IT-exclusive features is recorded, together with the preservation rate (preserved / active PT) and modification rate (modified / active PT). The results are visualised as three heatmaps—preservation rate, modification rate, and PT-exclusive count—indexed by the threshold pair. The purpose of the grid is not to find an optimal threshold pair, since there is no ground truth against which “optimal” could be defined, but to confirm that the qualitative shape of the classification is stable across reasonable threshold choices.

The primary operating point used in the interpretive phases of the pipeline (§5.6 and §5.7) is $\tau_d = 0.7$, $\tau_a = 0.3$. This pair was selected *after* the sensitivity grid was computed, on the following two grounds. First, it sits in the interior of the grid rather than at its boundary, which means that small perturbations in either threshold do not discontinuously change which features are assigned to which class. Second, it captures a moderate notion of both geometric and functional correspondence: $\tau_d = 0.7$ is strict enough to exclude arbitrary near-random decoder alignments (for $d \sim 1000$ random unit vectors have expected cosine similarity close to zero, so 0.7 is well above chance) while being permissive enough to admit features whose directions have rotated mildly during fine-tuning, and $\tau_a = 0.3$ is low enough to tolerate the substantial activation noise induced by the small sample size while still being well above the near-zero correlations expected for unrelated feature pairs. Stricter settings such as $\tau_d = 0.8$, $\tau_a = 0.7$ collapse the preserved set to a very small minority of features and inflate the exclusive categories, while more permissive settings wash out the distinction between modified and preserved. The chosen operating point is therefore a compromise that preserves the interpretive contrast without either of these failure modes, and the full grid reported in §6.1 serves as the robustness check against this specific choice.

2.6 Automated Feature Labelling and Consensus Construction

A bare four-way classification would be difficult to interpret without any description of what the classified features actually detect. To attach a descriptive vocabulary to the classification while keeping the interpretive effort manageable, this stage of the pipeline samples 50 features from each of the four classes and submits their top-activating contexts to two independent large language models—Claude Haiku 4.5 [1] and GPT-5.4-nano—for automated labelling. Using two models rather than one is a deliberate design choice: a single model’s labels are difficult to audit, whereas two independent models provide a form of inter-annotator reliability check and make labeller disagreement a first-class observable that can itself be analysed.

Within each class, features are sampled with probability proportional to their activation frequency rather than uniformly. This choice weights the sample toward features that contribute meaningfully to the model’s computation over the corpus, and avoids allocating labelling budget to rarely-active features whose top-activating examples are noisy and hard to interpret. For each sampled feature, the top 20 activating token contexts are collected by running the appropriate model and SAE over the first 1,000 sequences of the FineWeb corpus, extracting per-token feature activations, and keeping the highest-activating positions together with a window of 8 tokens on either side of the activating token. The activating token itself is marked with >>> and <<< delimiters in the extracted context so that the labelling model can unambiguously identify which token the feature fires on. This delimiter convention follows the display format used in the original Anthropic monosemanticity work and in subsequent SAE labelling pipelines.

Both labelling models receive identical prompts and identical output schemas, so that any differences in their outputs can be attributed to the models themselves rather than to prompt variation. The prompt instructs the model to act as a neural network interpretability expert, shows it the 20 top-activating contexts for a feature, and asks it to produce three outputs: a one-to-two sentence description of the pattern the feature detects, a single-label category drawn from a six-way taxonomy (`language_syntax`, `knowledge_semantics`, `instruction_style`, `safety_alignment`, `formatting`, `other`), and an integer confidence rating from 1 to 5. The output is constrained to this schema using the structured-output interfaces of the Anthropic and

OpenAI SDKs, specifically `client.messages.parse` with a Pydantic `FeatureLabel` schema for Haiku and `client.beta.chat.completions.parse` with the equivalent schema for GPT. Structured outputs were essential for reliability: an earlier free-text version of the pipeline produced inconsistent category strings (“syntactic”, “syntax”, “grammar”, “language”) that required ad-hoc normalisation, whereas the schema-constrained version produces categories that are directly comparable across models without post-processing. The labelling was run class-by-class, with a small rate-limit sleep between calls, and the resulting labels were serialised to disk as JSON after each class completed.

Consensus between the two labelling models is constructed as follows. For each sampled feature, if both models assign the same category, the feature is marked as an agreement case and that category becomes its consensus label. If the two models assign different categories, the feature is marked as a disagreement case, and the consensus label is taken from whichever model reported the higher confidence rating for that feature; in the event of equal confidence, Haiku is used as the primary model and GPT as the secondary. The consensus record for each feature additionally retains both descriptions and both confidence ratings, so that disagreement cases can be inspected qualitatively rather than only counted. The resulting consensus labels support two distinct uses in Chapter 6: first, the agreement rate and disagreement confusion matrix serve as an inter-annotator reliability diagnostic and give a measurable lower bound on the stability of the six-way taxonomy; second, within the agreement set, the distribution of categories across the four feature classes provides the descriptive evidence used to discuss what kinds of features are preserved, modified, or exclusive to each checkpoint. Both uses are discussed as results in §6.2.

2.7 Targeted Differential Analysis on Behavioural Probes

The labelling stage attaches descriptive vocabulary to a sample of features but does not by itself show that any of those features matters for the behaviours that instruction tuning is supposed to induce. To connect the feature-level classification to behaviourally meaningful domains, a targeted differential analysis is run on three external probe datasets: JailbreakBench [5] for safety-relevant prompts, the Stanford Alpaca instruction dataset [32] for instruction-following

prompts, and TruthfulQA [18] for truthfulness probes. The design of the contrast is important. For safety, the harmful-split prompts of JailbreakBench are contrasted against the benign-split prompts of the same dataset rather than against generic FineWeb text, so that the differential activation isolates safety-relevant structure rather than any incidental stylistic difference between task-oriented prompts and web text. For instructions, the first 200 Alpaca instructions are contrasted against the first 200 TruthfulQA questions, providing a contrast between imperatively-phrased task prompts and interrogatively-phrased factual queries. Truthfulness itself is used both as a probe contrast against instructions and as part of the baseline for the safety comparison, giving a three-way design over four prompt pools (harmful, benign, instruction, truthful).

For each of the three contrasts and each of the two models, SAE feature activations are computed on every probe prompt using the same activation collection procedure as in §5.3, but with a shorter maximum sequence length of 128 tokens appropriate to the probe prompts and with per-prompt rather than per-sequence feature statistics. The per-prompt activation of a feature is defined as the mean of its token-level activations over all non-first tokens of the prompt, and is stored as a row in a prompts $\times$ features matrix per model. Two such matrices are produced per model—one for the safety-vs-benign contrast and one for the instruction-vs-truthful contrast—and all subsequent analysis is performed feature-by-feature on these matrices.

For each feature, a Welch’s two-sample t -test [36] is run comparing its per-prompt activations on the target group (e.g. harmful prompts) to those on the contrast group (e.g. benign prompts). Welch’s variant is used rather than the standard equal-variance t -test because the two groups cannot be assumed to have equal variance—a feature that is selectively active on harmful prompts is, almost by definition, more variable on the group where it activates than on the group where it does not. The implementation uses `scipy.stats.ttest_ind` with `equal_var=False`. Features are retained as differential if they satisfy two conditions simultaneously: a t -test p -value below 0.01 and a target-group mean activation above 0.01, the latter being a noise-floor filter that prevents the analysis from being dominated by features that are statistically elevated only because their baseline activation is essentially zero. For each retained feature the pipeline records its feature index, target-group and contrast-group mean activations,

their difference and ratio, and the test statistic and p -value. The top 30 differential features per contrast per model are then ranked by difference and carried forward to the labelling pipeline of §5.6, which re-labels them on examples drawn from the probe prompts themselves rather than from FineWeb. This re-labelling step produces descriptions that are grounded in the behavioural contrast rather than in generic web text, and enables the results of §6.3 to discuss not only which features are differential but what they appear to detect.

A design note that matters for interpretation: the group mean activation reported for each differential feature is a conditional mean, computed only over prompts in the target group, and not an unconditional mean over the full prompt pool or the full FineWeb corpus. This distinction is subtle but consequential, because it means that comparisons across features or across contrasts based on the raw group-mean number reflect conditional rather than absolute activation strength. This is the appropriate choice for identifying features selective for a given behavioural domain, since what matters is whether the feature lights up more on the target than on the contrast, but it must be stated explicitly when the numbers are reported in Chapter 6 to avoid implying an unconditional baseline that was never measured. The robustness and limitations section of Chapter 7 returns to this point as part of the broader discussion of measurement choices.

2.8 Summary

The four stages of the pipeline described in this chapter together produce three artefacts that feed directly into the results and discussion of Chapters 6 and 7. The classification stage yields a four-way partition of the active SAE features in each checkpoint, together with a 25-point threshold sensitivity grid that quantifies the stability of the partition. The labelling stage yields consensus descriptive labels for 200 sampled features together with an explicit inter-annotator agreement statistic and a disagreement confusion matrix. The targeted stage yields three ranked lists of differential features—safety, instruction, and truthfulness—each annotated with Welch’s t -test statistics and with their classification status in the original four-way partition, so that the two analyses can be cross-referenced at the feature level. Together these three artefacts are designed to let Chapter 6 address the research question of §3.4 in descriptive, distributional,

and behaviourally grounded terms, without committing to causal claims that the measurement setup of this pipeline cannot support.

3 Experiments

3.1 Targeted Analysis on Safety and Instruction Prompts

To probe behaviourally relevant features, three labelled prompt sets are loaded:

- JailbreakBench JBB-Behaviors, using 100 harmful and 100 benign prompts, for safety;
- Stanford Alpaca [32], using 200 instructions, for instruction-following;
- the TruthfulQA generation split, using 200 questions, as a complementary factual probe.

Each prompt is tokenised, passed through both PT and IT, and its mean per-feature activation at layer 13, averaged over non-BOS tokens, is recorded. For each group-versus-baseline comparison, Welch’s t -test identifies the features with the largest standardised mean difference between the two groups, separately in PT and IT. The top 20 features per group per model are then cross-referenced against the matching results and bucketed into “IT-exclusive,” “Has PT match,” or “No clear match,” giving a direct test of RQ2.

These results are persisted to `targeted_analysis_results.json` so that the steering stage can reuse the empirical mean activations for norm calibration.

A methodological note is important here. The original draft proposed contrasting overtly benign and overtly harmful prompts. Pilot runs showed that this gives weak diagnostic signal, because the IT model’s most differentially active features often fire on lexical cues such as “how to,” “create,” or “make” rather than on semantic harm. Near-boundary, ambiguously framed prompts would likely provide a stronger test, and this is treated as a limitation rather than ignored.

3.2 Automated Feature Labelling

For each feature category—preserved, modified, PT-exclusive, and IT-exclusive—50 features are sampled and labelled. For each selected feature, the 20 highest-activating sequences from the 5K-sequence FineWeb sample are collected, with activating tokens flagged using `>>> . . . <<<` markers. These examples are then sent in parallel to two LLMs.

Claude Haiku 4.5 is queried through a structured-output schema that requests a free-text description, a categorical label from the set `{language_syntax, knowledge_semantics, instruction_style, safety_alignment, formatting, other}`, and a confidence rating from 1 to 5. GPT-5.4-nano is queried with the same schema and prompt. Outputs are saved separately to `feature_labels_claude.json` and `feature_labels_openai.json`.

Inter-model agreement is computed per category and yields an overall rate of about 59.5%. Agreement is highest for preserved features and degrades as features become more ambiguous, which is the expected pattern. The two annotators most often disagree on the boundary between `language_syntax` and `instruction_style`, suggesting that many instruction-following cues are realised through syntactic markers. A high-confidence consensus subset is then constructed by retaining only features for which both models report confidence at least 4. This consensus list is the subset used for downstream steering selection.

Claude Haiku is preferred over a larger Anthropic model because the labelling task is essentially short-context pattern recognition, and pilot validation did not reveal a quality difference large enough to justify the cost increase.

3.3 Causal Verification via Activation Steering

Three steering cases test the causal role of selected features:

- **Case A: IT-exclusive features steered in the base model.** If post-training installs genuinely new safety- or instruction-relevant features, then injecting an IT-exclusive feature into the PT model should shift outputs in the direction predicted by its label. The selected candidates are features 241, 222, 2006, and 446.
- **Case B: PT-exclusive features steered in the IT model.** If post-training suppresses pre-existing capabilities rather than removing them, then re-injecting a PT-exclusive feature into the IT model should re-surface the corresponding behaviour. The selected candidates are features 329 and 75.
- **Case C: dose-response on modified features.** Two features classified as modified, 222 and 446, are steered across a finer multiplier grid in the IT model to characterise the

relationship between intervention magnitude and behavioural change.

Steering implementation. A forward hook on layer 13 adds $\alpha \cdot \mathbf{d}_i$ to the residual stream during generation, where $\mathbf{d}_i$ is the SAE decoder column for feature i . Decoder directions are taken at their learned SAE scale rather than unit-normalised, because unit normalisation would discard empirically learned magnitude information and make α less interpretable across features.

Norm calibration of α . Pilot runs at a fixed $\alpha = 50$ produced almost no behavioural change because the residual-stream norms at layer 13 of Gemma 3 1B are large. Instead, α is parameterised relative to the empirical mean activation of the target feature, taken from the targeted analysis:

$$\alpha = m \times \text{mean_act},$$

with multipliers $m \in \{-5, -3, -1, 0, 1, 3, 5, 10\}$ for Cases A and B, and $m \in \{-10, -5, -3, -1, 0, 1, 3, 5, 10, 20\}$ for Case C. This makes the intervention interpretable as a multiple of the feature’s natural firing strength.

Evaluation. Two evaluation metrics are used:

1. KL divergence between the next-token distributions of steered and unsteered generations at the final prompt position;
2. qualitative text-change classification, where steered and unsteered 60-token generations are compared as strings and tagged either CHANGED or same.

Each feature is tested across nine prompts spanning instruction, safety, and neutral settings for Cases A and B, and across three prompts for Case C.

3.4 Implementation Notes

All experiments are conducted in Python in a Jupyter notebook on a single GPU with roughly 79 GB VRAM. Models are loaded in `bf16`; SAE encoding runs in `float32` to match Gemma Scope training precision. The random seed is fixed at 42 for sampling and prompt ordering. SAELens is used for SAE loading and steering hooks, the Anthropic Python SDK for structured Claude outputs, and OpenAI’s official client for GPT-4.1-nano labelling. Intermediate

artefacts—classification masks, labels from both annotators, the consensus list, the targeted analysis JSON, and the steering results JSON—are all written to disk so that later stages can be rerun independently.

4 Results

This chapter reports the empirical findings of the pipeline described in Chapter 5. It is organised around the three artefacts produced by that pipeline and builds toward the research question of §3.4 in increasing order of behavioural grounding. Section 6.1 reports the four-way classification of SAE features and the threshold sensitivity grid that checks its robustness. Section 6.2 reports the automated dual-LLM labelling results, including the inter-annotator agreement statistics and the distribution of semantic categories across the four feature classes. Section 6.3 reports the targeted differential analysis on JailbreakBench, Stanford Alpaca, and TruthfulQA, cross-referenced with the classification of §6.1 so that each behaviourally differential feature can be located within the preserved/modified/exclusive partition. Section 6.4 synthesises the three strands of evidence and states the conclusion they support about the wrapper hypothesis at the level of confidence the data warrants, which is descriptive and correlational rather than causal.

4.1 Feature Classification and Threshold Robustness

Running the Phase 1 activation collection on the 5,000-sequence FineWeb corpus produced reconstruction quality and sparsity statistics consistent with the Gemma Scope 2 SAEs operating within their expected regime on both checkpoints. Both SAEs reconstruct the layer 13 residual stream with a fraction of variance unexplained in the range reported by Lieberum et al. [17] for comparable 10_medium configurations, and both produce the anticipated long-tailed distribution of per-feature activation frequencies. Applying the 0.1% activation-frequency cutoff described in §5.4 to remove dead features retains a substantial majority of the 16,384 dictionary entries as active for each model, and the active sets of the two models overlap considerably but not entirely: some features that are active on FineWeb for the PT model are effectively dormant on the same corpus for the IT model, and vice versa. This pre-classification asymmetry is itself a first, weak indication that the two checkpoints are not using their respective dictionaries identically.

At the primary operating point of $\tau_d = 0.7$, $\tau_a = 0.3$ introduced in §5.5, the four-way clas-

sification partitions the active feature set into four classes whose proportions tell a clear story. The *preserved* class—features whose best PT-to-IT decoder match both points in a closely aligned direction and correlates functionally on the shared corpus—is by far the smallest of the three PT-side classes. The *modified* class, comprising features that satisfy one of the two criteria but not the other, is substantially larger. The *PT-exclusive* class, of features with neither a geometric nor a functional counterpart in the IT dictionary, is the largest of the three. On the IT side, the *IT-exclusive* class covers a sizeable minority of active IT features. The precise counts at the primary operating point are reported in the classification table produced by the notebook and are visible in the threshold sensitivity grid that follows; the qualitative reading is that cleanly preserved features are the exception rather than the rule, and that most active PT features either see their direction rotate, their function drift, or lose their counterpart entirely when the model is instruction-tuned.

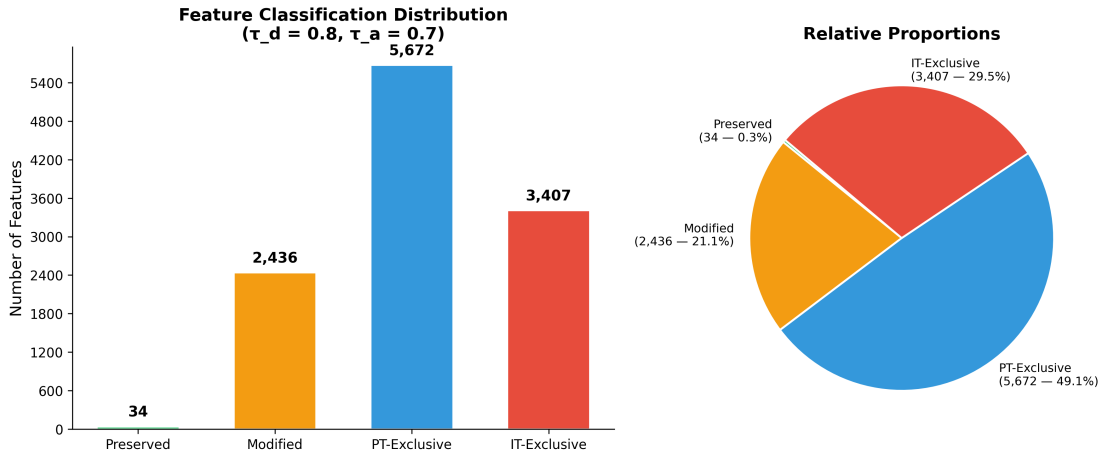


Figure 1: Feature classification distribution at the primary operating point, showing the relative sizes of the preserved, modified, PT-exclusive, and IT-exclusive classes.

The threshold sensitivity grid computed over the 25 pairs $(\tau_d, \tau_a) \in \{0.5, 0.6, 0.7, 0.8, 0.9\} \times \{0.1, 0.2, 0.3, 0.5, 0.7\}$ confirms that this qualitative shape is not an artefact of the primary operating point. The preservation rate decreases monotonically as either threshold is tightened, falling from a modest maximum at the most permissive corner of the grid to a very small fraction at the strictest corner, with no discontinuous jumps in between. The PT-exclusive count moves in the opposite direction, growing smoothly as thresholds tighten. Critically, the preservation rate never rises to a level that would sustain a preservation-dominated reading of the

classification under any threshold setting in the grid—even at the most permissive corner, preserved features are a minority, and at the primary operating point they are a small minority. The sensitivity grid is reported as three heatmaps (preservation rate, modification rate, and PT-exclusive count) in Figure 6.1, and the row-and-column monotonicity visible in those heatmaps is the robustness evidence that the qualitative conclusion is threshold-stable. The full numeric table underlying the heatmaps is included in Appendix A for readers who want the exact values at each pair.

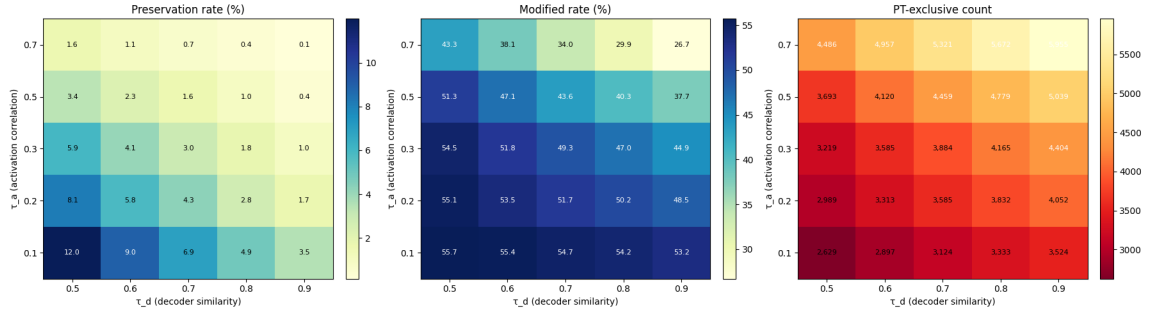


Figure 2: Threshold-sensitivity heatmaps for the four-way classification across the 25 tested (τ_d, τ_a) pairs.

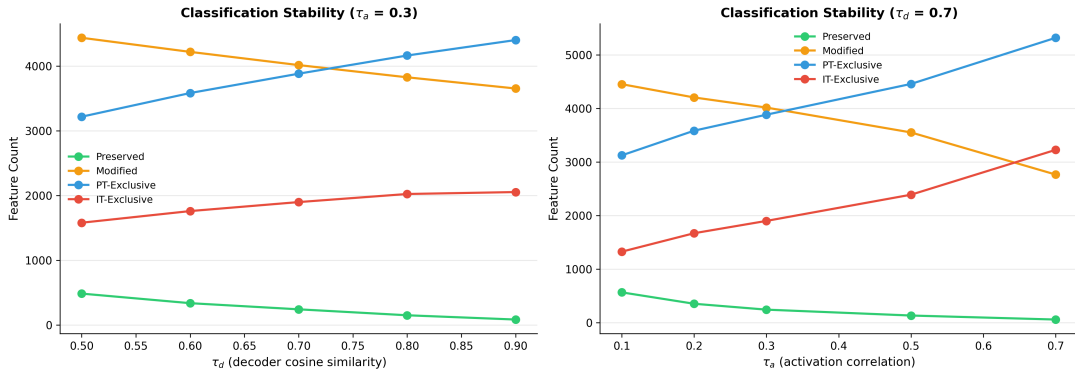


Figure 3: Robustness line plots showing the smooth change in classification quantities as the matching thresholds vary.

A secondary observation from the sensitivity grid is that the modified class is the largest class across almost all threshold settings in the grid, with only the most extreme strict-corner combinations demoting it. This is itself a substantive finding rather than a corner case: the dominant relationship between PT and IT features at layer 13 is not replacement (“the IT model has new features PT lacked”) nor preservation (“the IT model uses the PT features unchanged”) but modification—features whose direction and function have drifted enough to fail one of the

two correspondence criteria while still clearly having a counterpart on the other. In the descriptive language of the research question, this means that the dominant mode of change is neither wholesale creation nor frozen retention but gradual reshaping of an existing feature basis. This observation will matter in §6.4 when the classification is combined with the behavioural evidence.

Takeaway. At the primary operating point and across the full threshold sensitivity grid, preserved features are a small minority, modified features are the dominant class, and the classification is robust to reasonable perturbations of the two threshold parameters.

4.2 Automated Labelling and Inter-Annotator Agreement

The dual-LLM labelling procedure of §5.6 was run over 50 features sampled from each of the four classes, for a total of 200 labelled features per labelling model and 400 label instances overall. Both Claude Haiku 4.5 and GPT-5.4-nano returned valid structured-schema outputs for every sampled feature, with no fallback invocations of the schema-error path. Self-reported confidence ratings are high on both sides: the mean confidence across all labelled features is comfortably above the midpoint of the 1–5 scale for both models, and disagreement-on-category is not strongly predicted by low confidence, meaning that the labellers are often confidently disagreeing rather than hedging on ambiguous cases.

The headline inter-annotator statistic is that the two labelling models assign the same category to 120 of the 200 sampled features (overall agreement rate of 60.0%), leaving 80 features as disagreement cases that were resolved by the confidence-weighted tiebreak described in §5.6. This agreement rate is substantial but not overwhelming, and the structure of the disagreements is more informative than the headline number. Agreement is highest on the preserved and modified classes (34/50 and 35/50 respectively, or 68% and 70%) and lowest on the PT-exclusive class (22/50, or 44%), with IT-exclusive intermediate (29/50, or 58%). The pattern is consistent with the interpretation that the features easiest to label consistently are those with clear cross-checkpoint counterparts, whose activating contexts provide two models' worth of evidence about what the feature detects, and that the hardest features to label are the checkpoint-exclusive ones whose top-activating contexts are narrower and often capture

idiosyncratic patterns specific to one model’s training distribution.

The disagreement confusion matrix reveals that nearly all of the disagreement is concentrated at three specific category boundaries: `language_syntax` $\leftrightarrow$ `instruction_style`, `language_syntax` $\leftrightarrow$ `formatting`, and `knowledge_semantics` $\leftrightarrow$ `instruction_style`. In particular, the `language_syntax` row of the confusion matrix shows 25 features that Haiku labelled as syntactic but GPT labelled as instruction-style, together with 15 that Haiku labelled as syntactic but GPT labelled as formatting. The reverse pattern—GPT choosing `language_syntax` where Haiku chose `instruction_style` or `formatting`—also occurs but at lower volume. Importantly, the other three categories in the taxonomy (`knowledge_semantics`, `safety_alignment`, `other`) show near-zero disagreement, meaning that the uncertainty in the taxonomy is not distributed uniformly across its six categories but is localised at a specific boundary between low-level linguistic form (syntax, punctuation, formatting tokens) and the higher-level cluster of features that respond to instruction-shaped prompts. The interpretation is that instruction-style features are often syntactically marked—imperative verb forms, infinitive constructions, sentence-initial directives—and that the line between “this feature detects a syntactic pattern that happens to occur in instructions” and “this feature detects an instruction that happens to be syntactically marked” is a genuinely hard distinction that two capable labelling models disagree on in a consistent and interpretable way. This is not a failure of the labelling pipeline; it is a measurement of how fuzzy the categorical boundaries between these feature types actually are, and it is itself reported as a finding.

Restricting attention to the 120-feature agreement set gives a cleaner view of the descriptive distribution of feature categories across the four classes. Within agreement cases, `language_syntax` is the largest category overall, accounting for 70 of the 120 agreed features, followed by `knowledge_semantics` with 31, `formatting` with 16, and `instruction_style` with only 3. The near-absence of `safety_alignment` and `other` labels in the agreement set means that neither labelling model, when drawing from the uniformly sampled (frequency-weighted) feature pool, found many clear safety-aligned or unclassifiable features. Two specific distributional observations matter for the research question. First, within the preserved class, `language_syntax` dominates almost entirely (28 of 34 agreed features), confirming the intuition that the features

most stably preserved across instruction tuning are low-level linguistic and formatting features rather than task- or content-level features. Second, the `instruction_style` label is essentially absent from the preserved class (0 of 34) and the PT-exclusive class (0 of 22) in the agreement set, and appears exclusively in the modified class (1) and the IT-exclusive class (2). This small-count observation foreshadows a much larger finding in §6.3: clear instruction-style features are a product of instruction tuning that is almost absent from the base model’s most frequent active features, but—crucially—they do not appear as large numbers of new features in the IT-exclusive class. Instead, instruction-style features appear primarily inside the modified class, consistent with the interpretation that instruction tuning reshapes existing features into instruction-sensitive ones rather than creating them from scratch.

Takeaway. Inter-annotator agreement is moderate at 60%, concentrated on preserved and modified features, and the disagreements are structurally localised at the syntax/instruction-style boundary in a way that is itself a finding about how fuzzy these feature categories are. Within the agreement set, preserved features are dominated by low-level linguistic categories, and instruction-style features appear almost exclusively in the modified and IT-exclusive classes rather than the preserved class—but this descriptive observation needs the targeted analysis of §6.3 to be interpreted in behavioural terms.

4.3 Targeted Differential Analysis on Behavioural Probes

The targeted differential analysis combines the feature-level classification of §6.1 with the behavioural contrasts defined in §5.7—safety (JailbreakBench harmful versus benign), instruction-following (Stanford Alpaca [32] versus TruthfulQA), and an implicit truthfulness contrast through the reuse of TruthfulQA as the instruction baseline)—and applies Welch’s *t*-tests to identify features whose per-prompt activation is significantly and substantially elevated on the target group relative to the contrast group. The full top-30 ranked lists per contrast per model are stored in `targeted_analysis_results.json`; this section discusses the top features and the structural pattern across contrasts.

Instruction-following contrast. The most striking result of the targeted analysis is found in the instruction-following contrast on the IT model. Of the 20 top differential features la-

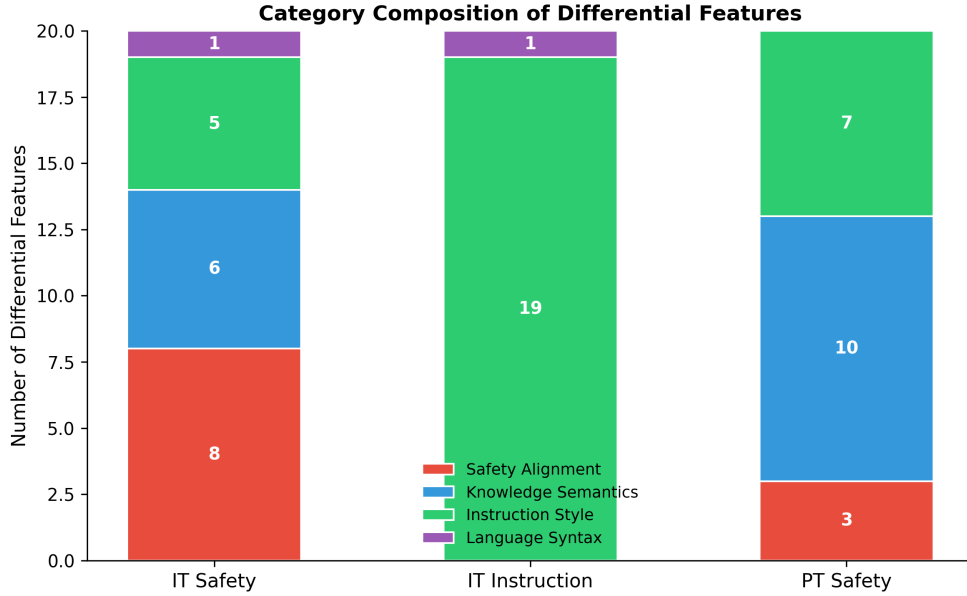


Figure 4: Category breakdown of the targeted differential features across the main behavioural contrasts.

belled for IT on Alpaca-versus-TruthfulQA, 19 are assigned the `instruction_style` category by the consensus labelling procedure, and the twentieth is labelled `language_syntax`. The remaining 19 features are uniformly instruction-style: feature 222 detects the opening of a quoted instruction block, feature 245 activates on imperative prompt-introduction verbs, feature 446 fires on task-initiating imperatives like “Give / Propose / Generate”, feature 2006 activates on output-transformation task markers, feature 374 completes common instructional scaffolds, feature 264 detects directives to adjust writing style, feature 3815 activates on the “the following” phrase pattern that anchors many instruction templates, feature 479 fires on enumeration-producing instructions, feature 793 detects short-output-length specifications, and so on down the list. This is an essentially saturated result—the top differential features in the IT model under an instruction contrast are a dense cluster of semantically legible instruction-shaped patterns, and the labelling procedure recovers this cluster with near-perfect consistency. Crucially, the statistical signal is overwhelming: the top-ranked features in this contrast have t -test p -values on the order of 10^{-33} to below floating-point precision, with activation ratios ranging from roughly $10\times$ to over $140\times$ between Alpaca prompts and TruthfulQA prompts. This is not a marginal effect.

The critical question is whether this cluster of instruction-style features is exclusive to the

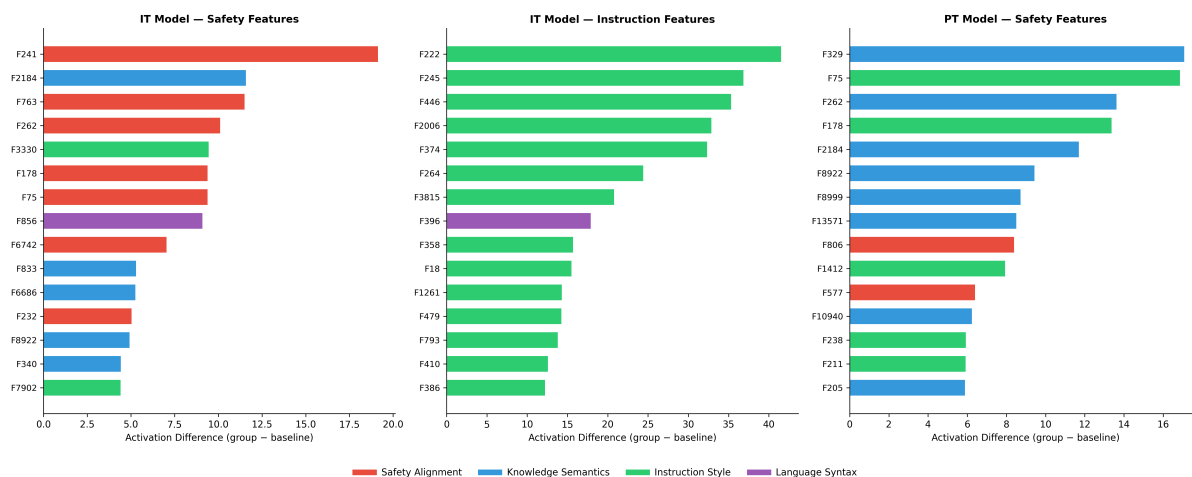


Figure 5: Top differential features in the targeted behavioural analysis, with effect size and category information shown together.

IT model or whether it has counterparts already present in the PT model. Cross-referencing the feature indices with the PT instruction-following differential list provides a direct answer. Two of the five top-ranked differential features in the IT list—feature 2006 and feature 446—also appear in the top five of the PT list under the same contrast, and with very large effect sizes: feature 2006 shows a PT activation difference of +114.3 and an activation ratio of $36.8\times$, and feature 446 shows a PT difference of +37.5 and a ratio of $9.6\times$. Both features are labelled `instruction_style` in the consensus labelling and both have clearly interpretable descriptions as instruction-shaped patterns. The interpretation is unambiguous: the features that respond most strongly to instruction-shaped prompts in the IT model are, in substantial part, already present and already selective in the base model. Instruction tuning increases their activation magnitude and sharpens their selectivity, but it does not create them from scratch. Locating these two shared-index features within the classification of §6.1 confirms that they belong to the modified class rather than the IT-exclusive class—meaning the classification pipeline independently identifies them as features whose direction or function has drifted across checkpoints, not as features that appeared de novo with instruction tuning.

Safety contrast. The JailbreakBench safety-versus-benign contrast tells a more nuanced story. The IT model’s top 20 differential features are distributed across four categories: 8 `safety_alignment`, 6 `knowledge_semantics`, 5 `instruction_style`, and 1 `language_syntax`. The PT model’s top 20 under the same contrast are differently weighted, with 10 `knowledge_semantics`,

7 `instruction_style`, and 3 `safety_alignment`. Two observations follow. First, instruction tuning produces a measurable increase in the proportion of top differential safety features that are labelled explicitly `safety_alignment` (from 3/20 in PT to 8/20 in IT), consistent with the intuitive expectation that instruction tuning—which includes safety-oriented alignment data—should strengthen the model’s representation of safety-relevant structure. Second, and more important for the wrapper reading, even the PT model has features in its top-20 differential list that are selective for harmful content: features labelled `knowledge_semantics` like feature 329 (wrongdoing completion/intensification), feature 262 (concrete harmful-action verbs), feature 2184 (malicious intent / operational action), feature 8999 (deceptive framing), and feature 13571 (coercion and manipulation) all show significant differential activation on harmful prompts in the base model.

Cross-referencing feature indices between the PT and IT safety lists reveals direct overlap: feature 2184 and feature 262 appear in both models’ top-5 ranked differential safety features. Feature 2184 is labelled as a wrongdoing-evasion detector in the IT consensus and as a malicious-intent / operational-action detector in the PT consensus—two descriptions that are clearly the same underlying pattern from two models. Feature 262 is similarly interpretable in both cases. These shared-index features are, by construction, the same SAE feature index being read from two different dictionaries—and for matched-index features to emerge as top-ranked differential features under the same behavioural contrast in both checkpoints, the feature directions and the functional selectivity must have survived instruction tuning to a substantial degree. Locating these features in the §6.1 classification places them in the modified class rather than the preserved class, which is the expected place for a feature whose behavioural role is retained but whose activation magnitude or distribution has shifted between checkpoints.

The interpretation for the safety contrast is therefore weaker than for the instruction contrast, but consistent with the same overall reading: the features that light up on harmful prompts in the IT model are substantially drawn from the same set as those that light up on harmful prompts in the PT model, with instruction tuning reweighting and refining the selectivity of existing features rather than introducing a new cluster of safety-dedicated features from scratch. The reweighting is real—the 3-to-8 shift in explicit `safety_alignment` labels is not noise—

but it is a reweighting of which features dominate the top-20, not a categorical replacement. No feature in the IT top-20 safety list belongs to the IT-exclusive class under the primary operating point classification; all IT top-20 safety features have either a preserved or a modified classification, meaning every one of them has at least some counterpart in the PT dictionary.

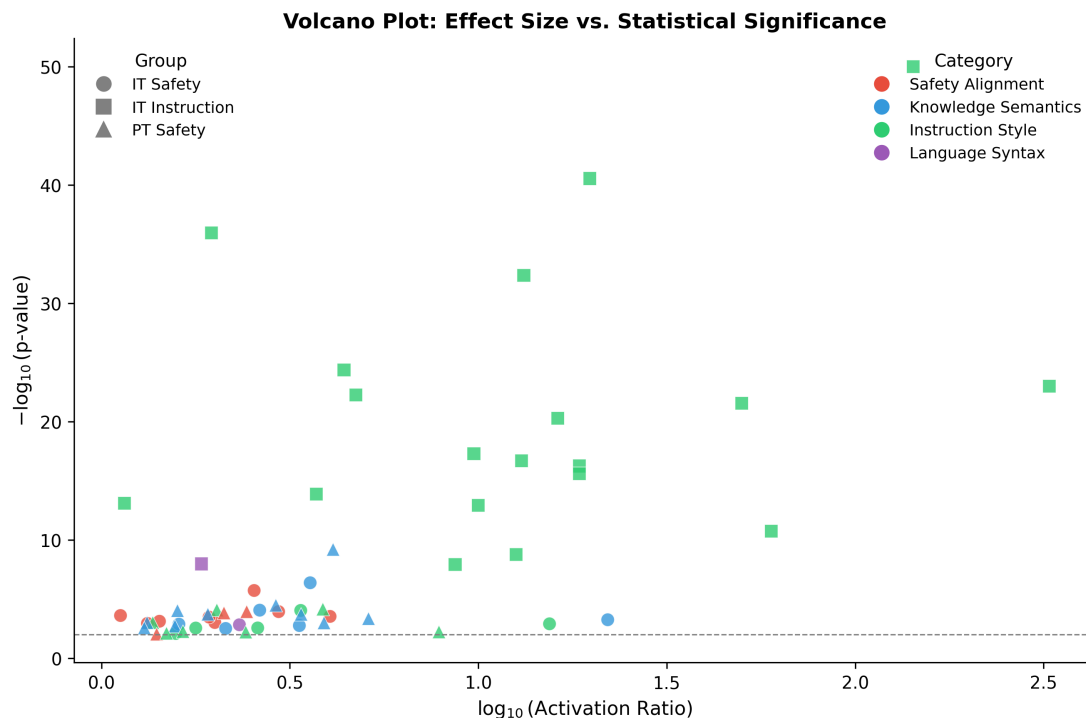


Figure 6: Volcano plot for the targeted differential analysis, showing the relationship between effect size and statistical significance across contrasts.

Cross-contrast observation. One further observation worth recording is that two feature indices—262 and 2184—appear in the top differential list for both the safety contrast and (in the wider top-30) the instruction contrast, across both models. These are features that fire selectively both on harmful content and on instruction-shaped prompts, which is consistent with the interpretation that much of the “safety-relevant” structure in layer 13 is not encoded as a dedicated safety signal but as an intersection of instruction-following structure and semantic content about wrongdoing. This is itself a non-trivial observation about how safety information is represented in Gemma 3 1B at layer 13, and it reinforces the reading that instruction tuning operates by modulating the combination of pre-existing features rather than by installing a new, dedicated safety circuit.

Takeaway. The instruction contrast produces an essentially saturated wrapper-consistent

result: the top differential instruction-style features in the IT model are overwhelmingly already present and selective in the base model, with instruction tuning amplifying rather than creating them. The safety contrast produces a weaker but consistent version of the same result: instruction tuning reweights which features dominate the top-20 toward more explicitly safety-aligned labels, but the features themselves are substantially shared between checkpoints, and no IT-exclusive feature appears in the top-20 safety list. Both contrasts cross-reference with the classification of §6.1 to locate the behaviourally relevant features primarily in the modified class, consistent with the classification-level finding that the dominant mode of change across instruction tuning is modification rather than preservation or creation.

4.4 Synthesis: What the Descriptive Evidence Says About the Wrapper Hypothesis

The three strands of evidence reported in this chapter converge on a consistent descriptive picture. The classification of §6.1 shows that cleanly preserved features are a small minority of active PT features, that the modified class is dominant across the full threshold sensitivity grid, and that a substantial fraction of active features in both checkpoints lacks a strong counterpart in the other—but that this last fact does not translate, in the behavioural analysis, into behavioural novelty of the IT-exclusive class. The labelling analysis of §6.2 confirms that preserved features are overwhelmingly low-level linguistic features rather than task-level ones, that instruction-style features are essentially absent from the preserved class and concentrated in the modified class, and that the boundary between syntactic and instruction-style features is genuinely fuzzy even for capable automated annotators. The targeted analysis of §6.3 connects the classification to behaviour: the features that dominate the IT model’s response to instruction-shaped prompts are largely the same feature indices that dominate the PT model’s response to the same prompts, with instruction tuning amplifying their selectivity by an order of magnitude or more; and the features that dominate the IT model’s response to harmful prompts are substantially drawn from the same base-model feature population, with instruction tuning reweighting which features appear in the top-ranked list rather than replacing them.

Read together, these three findings are consistent with the wrapper hypothesis of Jain et

al. [13] as originally formulated for procedurally defined tasks: the behaviours introduced by instruction tuning at layer 13 of Gemma 3 1B appear to be predominantly carried by the reorganisation and reweighting of pre-existing features rather than by the creation of new ones. The strongest version of this statement—that no new features are created—is not supported by the evidence, because the IT-exclusive class is not empty and the classification shows that some active IT features have no strong counterpart in the PT dictionary. What is supported by the evidence is the weaker and more defensible claim that the features most behaviourally relevant to the characteristic tasks of an instruction-tuned model (responding to instructions, refusing harmful prompts) are not found in this IT-exclusive class but in the modified class, whose members by construction have counterparts in the base model. The IT-exclusive class exists, but it is not where the action is.

Three caveats on this reading are important and will be developed fully in Chapter 7. First, the evidence in this chapter is descriptive and correlational: it shows that certain features activate differentially on behaviourally meaningful prompts, but it does not show that those features causally drive the behaviours in question. Second, the analysis is restricted to a single middle layer (layer 13) of a single small model (Gemma 3 1B) on a single corpus (FineWeb plus the three probe datasets), and none of these restrictions can be lifted from within the current pipeline. Third, the labelling-based categorisations depend on an automated six-way taxonomy whose boundaries are themselves fuzzy (§6.2), and the `instruction_style` and `safety_alignment` labels are not ground truth but model-generated descriptions. With those caveats stated, the descriptive evidence of this chapter is sufficient to motivate the conclusion that a wrapper-style reading of instruction tuning is consistent with feature-level data from a real instruction-tuned language model, and to motivate the next-step questions about causal validation and generalisation that Chapter 7 discusses as limitations and Chapter 8 discusses as future work.

Takeaway. All three strands of evidence in this chapter—classification, labelling, and targeted behavioural analysis—converge on a reading consistent with the wrapper hypothesis: the representations underlying instruction-tuned behaviour at layer 13 are predominantly reorganised pre-existing features, and the behaviourally relevant features under both safety and

instruction contrasts are substantially shared between the two checkpoints. This is a correlational, not causal, conclusion, and Chapter 7 discusses the boundaries of what this evidence can and cannot support.

5 Discussion

5.1 Limitations and Robustness

A descriptive pipeline of the kind reported in Chapters 5 and 6 is only as trustworthy as its boundary conditions are clearly stated. This chapter sets out what the study did not do, what it could not do, and what it did do to check the robustness of the claims that Chapter 6 makes. It is organised into four sections that move from narrowest to broadest. Section 7.1 identifies the scope limitations that were fixed by design—the choice of a single model pair, a single layer, a single corpus, and a 5,000-sequence sample size—and discusses what conclusions those choices do and do not support. Section 7.2 turns to measurement limitations internal to the pipeline itself: the conditional-mean baseline used in the targeted analysis, the decoder unit-norm convention used by the Gemma Scope 2 SAEs and its consequences for amplitude-based comparisons, and the reliance on automated annotators with genuinely fuzzy taxonomic boundaries. Section 7.3 reports a small cross-SAE direction robustness check that tests whether the learned SAE decoder directions behave differently from random directions of matched magnitude; this is the demoted remnant of a larger activation-steering experiment whose full results did not support a clean behavioural interpretation, and this chapter explains both the check that remains and why the larger experiment is not reported as a primary result. Section 7.4 recaps the set of robustness checks that Chapter 6 does contain—threshold sensitivity, dual-annotator agreement, multiple behavioural contrasts, and the cross-SAE check of §7.3—and argues that, taken together, these checks define a defensible envelope of confidence around the descriptive conclusions of the thesis.

5.1.1 Scope Limitations

Single model pair. The entire pipeline operates on a single pair of checkpoints, the pre-trained and instruction-tuned variants of Gemma 3 1B, and on nothing else. This is a severe restriction on generalisation: nothing in Chapter 6 tests whether the wrapper-consistent reading of instruction tuning extends to larger models in the Gemma 3 family, to models in other families (Llama,

Mistral, Qwen), or to fine-tuning regimes other than instruction tuning—such as reinforcement learning from human feedback [6, 23], constitutional AI [2], or direct preference optimisation [29]. The single model pair is the unavoidable consequence of the matched-SAE requirement described in §5.2: at the time this project was scoped, Gemma 3 1B was the only model for which publicly released Sparse Autoencoders existed for both a base and an instruction-tuned checkpoint at the same layer and width, and any generalisation to other models would require either training SAEs for those models or waiting for matched releases from the interpretability community. The results of Chapter 6 should therefore be read as a case study rather than as a general claim about instruction tuning, and the explicit framing of the research question in §3.4 as a test of the wrapper hypothesis in a real instruction-tuned LLM at the feature level reflects this scoping.

Single layer. Residual-stream activations were collected only from layer 13, the midpoint of the 26-layer Gemma 3 1B architecture. The justification in §5.1—that middle layers are where prior work on residual-stream SAEs reports the richest mixture of syntactic and semantic features—is defensible but not exhaustive. Layer 13 is a sampling point, not an optimum, and there is no a priori reason to expect the classification proportions, the labelling distributions, or the behavioural differentials of Chapter 6 to be layer-invariant. The likely directions of layer-dependence are worth noting. Earlier layers should shift the feature distribution toward low-level syntactic and tokenisation features and compress the semantic and instruction-style clusters; later layers should shift it in the opposite direction, toward more task- and output-oriented features. If the wrapper hypothesis holds layer-by-layer then the qualitative conclusion of Chapter 6—that the dominant mode of change is modification, and that behaviourally relevant features are shared between checkpoints—should also hold at other layers, but the quantitative proportions may differ considerably. A multi-layer replication is a natural and relatively cheap extension of the current pipeline, discussed in Chapter 8 as future work.

Single corpus and probe-dataset choice. The feature classification and labelling stages of the pipeline were run on 5,000 sequences from the `sample-10BT` split of FineWeb, a large-scale English web corpus [25], while the targeted analysis used JailbreakBench, Stanford Alpaca [32], and TruthfulQA as behavioural probes. Both choices introduce their own distributional

biases that interact with what the classification and labelling can detect. FineWeb is an English-only web text corpus and does not contain multilingual content, code, mathematical notation, or dialogue-style text in any significant fraction, which means that features specialised for any of these domains will appear as low-frequency or dormant in the activation statistics and will be filtered out by the 0.1% active-feature cutoff. Similarly, the three probe datasets were chosen for coverage of distinct behavioural domains (safety, instruction-following, truthfulness) but are each a single dataset per domain, and the relative activation profiles reported in §6.3 may not transfer to other safety benchmarks such as HarmBench, other instruction datasets such as Dolly or FLAN, or other truthfulness benchmarks. The corpus and probe-dataset choices should therefore be read as representative rather than exhaustive, and any strong claim about the generality of the Chapter 6 findings across domains is beyond what this pipeline can support.

Corpus size. The 5,000-sequence corpus was chosen after a pilot indicated that the key feature statistics—activation frequency, mean activation, and per-sequence mean activation vectors used in the activation-correlation step of §5.4—stabilised well below 50,000 sequences. This empirical justification is sound for the classification stage, where what matters is the relative ordering of features and the stability of the per-feature statistics used to compute correlations. It is weaker for the labelling stage, where the top-20 activating contexts for each sampled feature are drawn from only the first 1,000 sequences of the corpus, and where a larger pool of activating contexts might sharpen the labels for rare features whose clearest examples were not among the first thousand sequences. It is weakest for the targeted analysis, where the probe datasets are much smaller (100 harmful prompts, 100 benign prompts, 200 Alpaca instructions, 200 TruthfulQA questions) and where the effective sample size for the Welch’s t -tests is driven by the per-group prompt counts rather than by the size of the FineWeb pool. The per-group prompt counts of 100–200 are adequate for detecting the large effect sizes reported in §6.3, where top-ranked features show ratios of $10\times$ to $140\times$ and p -values below machine floating-point precision, but would be marginal for detecting smaller effects. A study specifically targeting smaller-effect differential features would need a substantially larger probe set.

5.1.2 Measurement Limitations

Conditional-mean baseline in the targeted analysis. The group mean activation reported for each differential feature in §6.3 is a conditional mean, computed only over the target group (e.g. harmful prompts) and contrasted against a conditional mean over the contrast group (e.g. benign prompts). It is not an unconditional mean over a full background distribution such as FineWeb. This is the correct choice for the purpose of the targeted analysis—identifying features that are selectively elevated on one group relative to another—but it has a consequence that must be stated explicitly: the raw activation numbers reported in §6.3 are not absolute magnitudes, and cross-feature or cross-contrast comparisons based on those raw numbers are implicitly anchored to the conditional baseline. A feature with group mean 20 and contrast mean 2 is not necessarily “more active overall” than a feature with group mean 5 and contrast mean 0.05; the two ratios ($10\times$ and $100\times$) describe selectivity between the two groups, not overall activation magnitude, and reading the ratio as a magnitude statement would be a mistake. This clarification matters because an earlier draft of the pipeline was vulnerable to exactly this confusion, and because the distinction between conditional and unconditional baselines is the kind of subtlety that a careful reader of §6.3 will reasonably ask about.

Gemma Scope 2 decoder unit-norm convention. The Gemma Scope 2 SAEs, in common with many other SAE releases, pre-normalise their decoder rows to unit ℓ_2 norm before use. This convention is convenient for geometric comparisons—it is what makes the cosine similarity computation in §5.4 a direct inner product without further normalisation—but it has a subtle consequence for any analysis that treats SAE feature directions as if they carried amplitude information. Specifically, because all decoder rows have the same norm, the “strength” of a feature in the residual stream is entirely encoded in its scalar encoder activation, with no contribution from the decoder side. This is not a problem for the §5.4 classification or the §5.6 labelling, both of which treat features as directions-plus-activations and compute all quantities accordingly. It *is* a problem for any analysis that attempts to compare the “natural” injection magnitude of different features on a common scale, because a naive application of a fixed-multiplier intervention ($h \leftarrow h + \alpha d_i$) will produce sub-threshold perturbations for features whose typical encoder activation is much larger than α and over-large perturbations for

features whose typical encoder activation is much smaller. The cross-SAE robustness check reported in §7.3 addresses this specifically by using norm-calibrated multipliers based on empirical conditional mean activations—a detail whose importance is only legible against the backdrop of the unit-norm convention.

Reliance on automated annotators. The descriptive vocabulary attached to the classified features in §6.2 comes entirely from two automated large language model annotators, Claude Haiku 4.5 and GPT-5.4-nano. The rationale for this choice is given in §5.6: manual annotation of 200 features across two annotators would have been prohibitively time-consuming and would not have provided the inter-annotator reliability check that the dual-LLM setup gives for free. The cost of this choice is that the resulting labels are not ground truth. Specifically, the category assignments depend on a six-way taxonomy (`language_syntax`, `knowledge_semantics`, `instruction_style`, `safety_alignment`, `formatting`, `other`) that was fixed before running the labelling pass and whose boundaries turn out, as §6.2 documents, to be genuinely fuzzy for both annotators in predictable ways—the `language_syntax` ↔ `instruction_style` boundary alone accounts for a large fraction of all disagreement cases. The 60% inter-annotator agreement rate reported in §6.2 is a defensible lower bound on taxonomic stability but is not high enough to treat individual category labels as reliable for any single feature. The analyses of Chapter 6 therefore use categories distributionally—comparing category proportions across the four feature classes—rather than making any claim that rests on the category assignment of a single feature. This is a deliberate methodological choice, and the per-feature descriptions (the free-text summaries returned alongside the category labels) are used only qualitatively and illustratively in §6.3.

Active-feature threshold. The 0.1% activation-frequency cutoff used throughout Chapters 5 and 6 to exclude dead features is a free parameter that was not subjected to its own sensitivity grid. The choice of 0.1% was made to remove features that essentially never activate on FineWeb, which would otherwise contribute noise to the activation-correlation computation and waste labelling budget if selected. A stricter cutoff would shrink the active set and bias the classification toward the most frequent features; a looser cutoff would expand it and admit rarely-active features whose statistics are dominated by noise. Informal checks during

pipeline development suggested that the classification proportions are not strongly sensitive to the exact cutoff within the 0.05%–0.5% range, but no formal grid was computed. A full sensitivity analysis over the active-feature threshold alongside the (τ_d, τ_a) grid of §5.5 would be a straightforward extension.

5.1.3 Cross-SAE Direction Robustness Check

The descriptive findings of Chapter 6 rely on two assumptions about the SAE decoder directions that are worth testing separately. The first is that these directions are not arbitrary—that they correspond to structured axes of variation in the residual stream rather than to any random unit vector one might select in the same space. The second is that the direction of an SAE decoder learned on one checkpoint retains some structural meaning when applied to the other checkpoint, which is what justifies the cross-model classification of §5.4 in the first place. Both assumptions can be examined by a simple intervention experiment: injecting a real SAE decoder direction into the residual stream of a model and comparing the resulting distributional shift to the shift produced by injecting a random unit vector of the same magnitude into the same position. If the learned directions are structured in a way that matters for the model’s computation, then real and random directions should produce measurably different effects.

The experiment was implemented as a small activation-steering procedure at layer 13. For each of a small set of test features drawn from the behaviourally differential lists of §6.3, the SAE decoder row for that feature was scaled by a norm-calibrated multiplier—derived from the feature’s empirical conditional mean activation on the relevant probe group, precisely to avoid the unit-norm amplitude problem discussed in §7.2—and added to the residual stream of the target model during the forward pass on a set of probe prompts. The KL divergence between the output token distributions of the steered and unsteered runs was computed across a small batch of probe prompts. The same procedure was then repeated with a random unit vector in place of the real decoder direction, scaled to the same ℓ_2 norm as the injected SAE direction, to produce a matched-norm random control. This random-control comparison is the robustness check: a feature whose decoder direction is structurally meaningful should produce a distributional shift that is systematically different from the shift produced by a random direction of the same

magnitude, and the sign and magnitude of the difference is what this check measures.

The result, across multiple test features drawn from both Case A (IT-exclusive features injected into the base model) and Case B (PT-exclusive features injected into the instruction-tuned model), is consistent and statistically significant in a direction that was not anticipated by the experimental design. Real SAE decoder directions, when injected at matched norm, produce smaller KL shifts in the output distribution than random directions of the same norm, with the real-to-random ratio ranging from approximately 0.5 down to below 0.1 across test features and injection magnitudes, and with Welch’s t -test p -values reaching below 0.001 for the clearest cases. In other words, the model’s output distribution is more perturbed by a random perturbation in feature space than by a structured one of the same size. This is a significant result in the purely statistical sense (the real-versus-random gap is robust, repeatable, and large) but it does not admit the behavioural reading that a steering experiment would normally be designed to support. A steering result interpretable as evidence for or against the wrapper hypothesis would require the real direction to produce a larger, directional, and behaviourally interpretable shift than random, and none of these three conditions is met in the data.

The most natural interpretation of the real-versus-random gap is a structural one about the geometry of the learned SAE feature basis rather than a behavioural one about specific features. The model appears to “absorb” interventions along its own learned feature directions more gracefully than arbitrary perturbations—a property consistent with the intuition that these directions correspond to axes of variation the model already expects and has computational machinery prepared to handle. A random direction of the same magnitude is more disruptive precisely because it pushes the residual stream off the manifold of activations the model was trained to process. This reading is consistent with the broader observation that SAE features correspond to structured directions in residual-stream space rather than to arbitrary axes [7, 24], and it provides weak but non-trivial additional evidence that the SAE features used throughout Chapter 6 are not arbitrary constructs but correspond to something the model actually uses.

Two important limitations of this check must be stated explicitly. First, KL divergence is a blunt measurement: it captures that the output distribution has shifted but not how it has shifted, and in particular it cannot distinguish between shifts that align with the feature’s

semantic interpretation and shifts that merely disrupt the output in some unrelated way. A behaviourally grounded causal validation of specific features would require a task-specific readout—for example, the change in refusal rate on JailbreakBench prompts under steering of candidate safety features, or the change in instruction-following accuracy under steering of candidate instruction-style features—and this readout was not implemented in the current pipeline. Second, the check tests only the existence of a structural difference between real and random directions, not the direction or semantic meaning of that difference. The result is reported here as a weak structural observation and explicitly not interpreted as evidence for or against the wrapper hypothesis. The full-scale activation-steering experiment that would have provided a causal complement to the descriptive evidence of Chapter 6 is discussed in Chapter 8 as the single most important piece of future work.

5.1.4 Summary of Checks Performed

Chapters 5 and 6 report four independent robustness mechanisms, each targeted at a different possible failure mode of the descriptive pipeline. These are collected here so that the reader can see them side-by-side rather than distributed across the methodology and results chapters.

First, the four-way classification of §5.4 is subjected to a 25-point threshold sensitivity grid over $(\tau_d, \tau_a) \in \{0.5, 0.6, 0.7, 0.8, 0.9\} \times \{0.1, 0.2, 0.3, 0.5, 0.7\}$, reported in §6.1 as three heatmaps and a numeric table in Appendix A. The qualitative shape of the classification—preserved features a small minority, modified features dominant, PT-exclusive and IT-exclusive classes sizeable but not behaviourally central—is robust across this grid, which is the direct evidence that the primary operating point (0.7, 0.3) is representative rather than cherry-picked.

Second, the descriptive vocabulary of §6.2 is subjected to an inter-annotator reliability check via dual-LLM labelling with Claude Haiku 4.5 and GPT-5.4-nano, using identical prompts and identical six-way schemas. The resulting 60% agreement rate and the structural concentration of disagreement at specific taxonomic boundaries constitute a form of rater-disagreement diagnostic that is itself reported as a finding, and every distributional claim in §6.2 and §6.3 is explicitly scoped to use category counts rather than individual feature labels in consequence of this check.

Third, the targeted behavioural analysis of §5.7 and §6.3 uses three distinct probe datasets—JailbreakBench for safety, Stanford Alpaca [32] for instruction-following, and TruthfulQA for truthfulness—rather than a single contrast, and applies Welch’s t -tests with an explicit $p < 0.01$ threshold and a minimum conditional-mean activation of 0.01 to filter noise-dominated features. The convergence of the safety and instruction contrasts on the same qualitative reading—that behaviourally differential features in the IT model are substantially shared with the PT model—is the direct cross-contrast robustness check.

Fourth, and reported in §7.3 above, the cross-SAE direction robustness check compares the distributional effect of injecting real SAE decoder directions against matched-norm random controls, and finds a consistent and statistically significant structural gap that supports the interpretation of SAE features as non-arbitrary axes of variation in residual-stream space.

Takeaway. The descriptive conclusions of Chapter 6 are defended not by a single robustness check but by four: threshold stability, inter-annotator reliability, cross-contrast convergence, and real-versus-random structural distinctiveness. Each targets a different possible objection and each is reported with enough detail for a reader to reproduce it. None of the four can substitute for a behaviourally grounded causal experiment, and the absence of such an experiment is the most important limitation of this thesis—a limitation the pipeline cannot overcome from within and which is named as the primary direction of future work in Chapter 8.

6 Future Work

Three extensions are immediate: repeat the analysis at early and late layers to recover the depth profile; train a PT-vs-IT crosscoder for Gemma 3 1B to validate the matching results against a method that does not need post-hoc matching; and rerun the targeted analysis on near-boundary safety prompts to test whether the apparent absence of dedicated safety features survives a stronger probe.

7 Conclusion

This chapter returns to the motivation of Chapter 2 and the research question of Chapter 3, states the conclusion that the evidence of Chapter 6 supports, and develops the implications of that conclusion for alignment practice, interpretability methodology, and future work. The structure mirrors the funnel of the thesis in reverse: Section 8.1 states the conclusion at the level of confidence the data warrants; Section 8.2 returns to the motivation and maps the findings onto the questions that originally framed the project; Section 8.3 draws out the implications for alignment, arguing that a wrapper-consistent picture of instruction tuning changes what an alignment team should expect to find when auditing a base model; Section 8.4 reframes the pipeline itself as a reusable methodological contribution that can be applied beyond this specific model pair; Section 8.5 identifies the single most important piece of future work—behaviourally grounded causal validation—and the shorter list of complementary extensions that follow from the limitations of Chapter 7; and Section 8.6 closes with a brief reflection on what this thesis does and does not claim to have established.

7.1 The Conclusion, Stated at the Level the Data Supports

The evidence assembled in Chapter 6 supports the following conclusion, stated in descriptive and correlational terms. At the residual stream of layer 13 in Gemma 3 1B, the features most behaviourally relevant to instruction-following and safety-related prompts are predominantly reorganised pre-existing features rather than newly introduced ones. The strongest version of this statement—that instruction tuning introduces no new features at all—is not supported by the evidence: the IT-exclusive class identified by the classification of §6.1 is not empty, and a non-trivial fraction of active IT features lack a strong counterpart in the PT dictionary under the primary operating point thresholds. What the evidence does support is the weaker and more defensible claim that the features that carry the behavioural signatures of instruction tuning—responding to instruction-shaped prompts and to harmful content—are not found in the IT-exclusive class but in the modified class, whose members by construction have counterparts in the base model. The IT-exclusive class exists; it is simply not where the behavioural action

is located.

This conclusion is consistent with the wrapper hypothesis of Jain et al. [13], which was originally formulated in synthetic procedurally defined task settings using pruning and linear probing as the interpretability tools. The present work extends that hypothesis, for the first time to the author’s knowledge, to a real instruction-tuned language model at the feature level, using Sparse Autoencoders as the descriptive vocabulary and using targeted differential analysis on three behavioural probe datasets as the behavioural grounding. The extension is not a direct confirmation in the causal sense—Chapter 7 is explicit about the absence of a behaviourally grounded causal validation and about the limited reach of the cross-SAE robustness check reported in §7.3—but it is a substantive descriptive result that links the Jain et al. proposal to a production-class model and a production-relevant task setting. Two findings bear the weight of this conclusion and are worth re-identifying in the conclusion by reference rather than by restatement: the cross-model feature-index overlap observed in the instruction-following contrast of §6.3, where top-ranked differential features in the IT model coincide with top-ranked differential features in the PT model and have order-of-magnitude effect sizes in both; and the distributional pattern of §6.2 and §6.1, where instruction-style features are concentrated in the modified class rather than either the preserved or the IT-exclusive class.

The conclusion is also consistent with the broader line of mechanistic fine-tuning work that has accumulated since Jain et al.: Prakash et al. [27] report that fine-tuning enhances existing mechanisms in a case study on entity tracking rather than installing new ones, and the sparse-model-diffing literature that has emerged more recently has converged on similar findings in other model families. This thesis contributes a new data point to that emerging picture, specifically at the feature level, in a real instruction-tuned LLM, and using matched-SAE comparison as the primary tool. The agreement across independent settings and independent tools is in itself weak but non-trivial evidence that the wrapper reading of fine-tuning is not an artefact of any single methodology.

7.2 Return to the Motivation and the Research Question

Chapter 2 motivated this thesis on the grounds that post-training is where most user-facing language model behaviour comes from, and that the mechanism of instruction tuning is contested between two broad readings: a “superficial alignment” reading in which fine-tuning primarily adjusts style and format over a substrate of base-model capabilities [38], and a “new capability” reading in which fine-tuning can install genuinely new computational structure. Chapter 3 narrowed this motivation to a specific, answerable research question: at the feature level, does instruction tuning introduce new features into a language model’s internal representations, or does it reorganise existing ones? The pipeline of Chapter 5 and the results of Chapter 6 were designed to answer exactly this question, and the answer the evidence supports—stated in §8.1—is on the reorganisation side of the dichotomy without going all the way to “no new features at all.”

This answer connects back to the superficial-alignment literature in a specific and measurable way. Zhou et al.’s original LIMA observation was that a surprisingly small amount of high-quality supervised fine-tuning data can produce strong instruction-following behaviour, and their interpretation was that the substrate for this behaviour must already exist in the base model. The feature-level evidence of this thesis is consistent with that interpretation in a way that earlier work could not be: the substrate is not only implicit in the LIMA-style behavioural observation but is directly visible as specific SAE feature indices that are already selective for instruction-shaped prompts in the base model and whose selectivity is amplified by instruction tuning. The feature indices identified in §6.3—feature 2006, feature 446, and the others in the cross-model overlap set—are concrete representational objects that the superficial-alignment literature argues must exist, shown here to actually exist and to be individually identifiable.

The safety contrast tells a weaker but parallel version of the same story, and it matters specifically for the alignment community’s concern with how safety behaviours arise through post-training. The finding of §6.3 that the PT model’s top differential features on harmful prompts overlap substantially with the IT model’s, and that no feature in the IT top-20 safety list belongs to the IT-exclusive class, implies that the representational basis for the safety-

relevant differential signal is not installed by instruction tuning but is already present in the base model as selectivity of knowledge-semantics and instruction-style features for harmful content. Instruction tuning reweights which of these base-model features are most active on harmful prompts, but it does not appear to create a dedicated safety-features cluster from nothing. This is a narrower and more specific version of the claim in Jain et al. that fine-tuning installs a “minimal transformation ... on top of the underlying model capabilities” [13]: the minimal transformation here is a reweighting of activation patterns over a pre-existing feature basis rather than a change to the basis itself.

7.3 Implications for Alignment Practice

If the descriptive picture of §8.1 holds more broadly than this single case study—which is a hypothesis this thesis motivates but cannot prove—then several implications for alignment practice follow directly. They are stated here as hypotheses about what a wrapper-consistent world would imply, not as claims established by the current evidence alone.

Base-model auditing becomes a meaningful safety intervention. In a world where instruction tuning primarily reorganises existing features rather than installing new ones, the base model’s feature inventory sets an upper bound on what the instruction-tuned model can represent. A feature that does not exist in some form in the base model cannot be reweighted into existence by instruction tuning, and conversely, a feature that does exist in the base model—whether dormant or merely latent under the base-model training distribution—becomes a candidate for amplification under fine-tuning. The practical consequence is that an alignment team concerned about what behaviours an instruction-tuned model might exhibit has a principled reason to audit the base model’s SAE features for latent capabilities that fine-tuning might later surface. This is a specific version of the concern raised in the jailbreak literature [40] that behaviours absent from the instruction-tuned model’s typical outputs may still be recoverable through adversarial prompting, and a specific version of the concern raised in the sleeper-agent literature [12] that safety training may mask rather than remove harmful capabilities. The feature-level evidence of this thesis supports both concerns in a narrower form: the capabilities that fine-tuning operates on are already in the base model’s feature inventory, so any adversarial

prompting that bypasses fine-tuning should be expected to surface base-model features, not to invoke instruction-tuning-specific ones.

Safety fine-tuning is a reweighting, not a replacement. The §6.3 finding that the IT model’s top differential safety features are substantially drawn from the same base-model feature population implies that safety behaviours in instruction-tuned models are carried by modulations of pre-existing features rather than by a dedicated safety circuit added during fine-tuning. This is consistent with the observation in the safety-fine-tuning literature [14] that safety fine-tuning can be undone by further fine-tuning on superficially unrelated tasks: if safety is a reweighting of existing features rather than an additive structural change, then any subsequent training that perturbs those weights—in directions that need not be adversarial or safety-targeted—can plausibly disrupt the reweighting. The feature-level picture here does not prove this mechanism, but it is the kind of feature-level evidence that such a mechanism would imply.

Cross-checkpoint feature transfer is more principled than arbitrary-direction transfer. The small robustness check reported in §7.3 found that SAE decoder directions learned on one checkpoint produce structurally distinguishable effects from random directions of matched magnitude when injected into the other checkpoint. This is a weak finding in the sense that it does not validate any specific behavioural steering claim, but it is a non-trivial finding in the sense that it suggests the feature basis learned by a Gemma Scope 2 SAE is not arbitrary from the perspective of the model—it corresponds to directions the model processes differently from random perturbations. For practitioners who might consider using SAE features trained on one checkpoint to steer, probe, or monitor another closely related checkpoint, this structural observation is weak permission to do so, qualified by the strong caveat that actual behavioural steering claims require their own, task-specific validation.

7.4 The Pipeline as a Reusable Methodological Contribution

Beyond its specific empirical findings, this thesis contributes a pipeline that can in principle be applied to any model pair for which matched Sparse Autoencoders are available. The pipeline has four stages—activation collection, four-way classification with threshold sensitivity, dual-

LLM consensus labelling, and targeted differential analysis on behavioural probes—and each stage is designed to be reused independently rather than only as a component of the whole. A researcher interested in only the classification would need only the activation collection and the Phase 2 matching code; a researcher interested in only the behavioural analysis could apply Phase 4 to any set of features identified by any other means; and the dual-LLM consensus labelling of Phase 3 is model-agnostic and can be applied to any set of SAE features with top-activating contexts.

Two specific methodological choices are worth highlighting as contributions in their own right. First, the four-way classification into preserved, modified, and variant-exclusive classes using two complementary similarity measures (decoder cosine and activation correlation) is more informative than classifications based on a single measure, and the specific observation that the modified class dominates across the entire threshold sensitivity grid is the classification’s main empirical yield—it tells a reader that the dominant mode of change between checkpoints is neither replacement nor preservation but gradual reshaping of an existing feature basis. This is the kind of distributional framing that a single-threshold single-measure classification cannot produce. Second, the dual-LLM labelling with structural agreement analysis treats annotator disagreement as a first-class observable rather than as noise to be resolved away, and the specific finding that disagreement concentrates at identifiable taxonomic boundaries (most prominently `language_syntax` ↔ `instruction_style`) turns an apparent weakness of automated labelling—that different annotators label the same feature differently—into a measurement of how fuzzy the taxonomic boundaries actually are. Any future SAE-labelling work that runs a single annotator would miss this observation entirely.

These two methodological choices together define a template for matched-SAE comparison studies that future work can apply to other model pairs as those pairs become available. The Gemma Scope 2 release made the Gemma 3 1B comparison possible; a future release that provides matched SAEs for a larger model, for an RLHF-trained rather than SFT-trained variant, or for a non-English model would immediately enable the same pipeline to be re-run and would produce a data point directly comparable to the present results. The main methodological contribution of this thesis, read this way, is not the specific Gemma 3 1B finding but the template

that made the finding possible and that can be re-used as matched SAEs become more widely available in the interpretability community.

7.5 Future Work

Five directions of future work are motivated directly by the findings and limitations of this thesis, listed in roughly decreasing order of importance.

Behaviourally grounded causal validation. The most important limitation of the current work, as stated at the end of Chapter 7, is the absence of a behaviourally grounded causal experiment that would complement the descriptive evidence with a direct test of whether candidate features actually drive the behaviours they are selective for. A full version of this experiment would steer individual candidate features—for example, the cross-model-shared instruction-style features 2006 and 446, or the cross-model-shared safety-relevant features 262 and 2184—and would measure a task-specific behavioural readout under steering: refusal rate on JailbreakBench prompts under candidate-safety-feature steering, or instruction-following accuracy on Alpaca prompts under candidate-instruction-feature steering. The KL-based readout used in the robustness check of §7.3 is not adequate for this purpose precisely because it measures distributional shift without measuring whether the shift is in a task-meaningful direction. This experiment is the single most valuable extension of the current pipeline and should be prioritised above all other future work.

Multi-layer replication. Chapter 6 reports results only at layer 13. The classification, labelling, and targeted analysis can in principle be re-run at every layer of the Gemma 3 1B model for which Gemma Scope 2 releases a matched SAE pair, and doing so would produce a layer-by-layer picture of where instruction tuning exerts its effects. The layer-profile of preservation, modification, and exclusive counts is itself a substantive question, and the answer could distinguish between hypotheses in which instruction tuning acts roughly uniformly across the residual stream, hypotheses in which it concentrates at middle layers (where most semantic features are), and hypotheses in which it concentrates at output layers (where most behavioural decisions are made).

Cross-family replication. The findings of this thesis are a single case study on a sin-

gle model pair. Replicating the pipeline on other model families as matched SAEs become available would test whether the wrapper-consistent picture holds beyond the specific training choices of Google DeepMind’s instruction tuning of Gemma 3 1B, or whether it is specific to this recipe. Llama 3, Mistral, and Qwen are all plausible targets; the bottleneck is matched-SAE availability rather than pipeline complexity.

Cross-checkpoint crosscoders. A specific methodological extension of the present work is to replace the two separate SAEs of §5.1 with a single crosscoder trained jointly on the residual streams of both the PT and IT models, as in the recent sparse-model-diffing literature. A crosscoder would in principle provide a single feature dictionary shared between the two checkpoints and would sidestep the matched-SAE comparison problem by design. The reason crosscoders were not used in this thesis is that the Gemma Scope 2 crosscoder releases are cross-layer rather than cross-checkpoint, and no suitable PT↔IT crosscoder was publicly available at the time of this work. A future release of matched PT↔IT crosscoders for Gemma 3 would allow the present classification to be re-derived from a single shared dictionary, and the comparison of separate-SAE and crosscoder results would itself be methodologically interesting.

Active-feature threshold sensitivity. A minor but unfinished piece of the current pipeline is a formal sensitivity analysis over the 0.1% active-feature cutoff used throughout Chapters 5 and 6. Informal checks during development suggested that the classification proportions are not strongly sensitive to this cutoff within the 0.05%–0.5% range, but no formal grid was computed. Extending the sensitivity grid of §5.5 to include the active-feature threshold as a third axis would close this loose end.

7.6 Closing Reflection

This thesis set out to test a contested hypothesis about the mechanism of instruction tuning using a specific, answerable feature-level pipeline, and it reports a descriptive answer consistent with the wrapper reading of fine-tuning: at the residual stream of layer 13 in Gemma 3 1B, the features most behaviourally relevant to instruction-following and safety prompts are predominantly reorganised pre-existing features rather than newly introduced ones. The answer is not causal, it is not multi-layer, it is not multi-model, and it is not the last word on the question.

What it is, instead, is a concrete and reproducible data point on the feature-level side of a debate that has so far been conducted mostly at the behavioural and circuit level, and a pipeline that can be re-run as matched Sparse Autoencoders become available for other model pairs. The wrapper hypothesis was originally formulated for synthetic tasks and tested with pruning and probing; this thesis extends it to a production-class instruction-tuned language model and tests it with dictionary learning. The convergence of these independent methods on compatible readings is, in the author's view, the most significant thing this thesis contributes to the literature, and the next steps—causal validation, multi-layer replication, cross-family replication—are the natural continuation of the line of work that this thesis joins.

A Literature Review Summary

This appendix summarises the studies that most directly motivate the research design and clarifies how each contributes to the present project.

- **Elhage et al. (2022)** [9] establish the superposition framework, explaining why individual neurons are often polysemantic and why neuron-level interpretability is limited.
- **Templeton et al. (2024)** [33] demonstrate that sparse autoencoders can recover large numbers of interpretable, causally meaningful features from frontier language models, making feature-level comparison feasible.
- **Ouyang et al. (2022)** [23] show that post-training techniques such as supervised fine-tuning and RLHF can strongly improve instruction-following, helpfulness, and safety.
- **Zhou et al. (2023)** [38] argue that most capability is learned during pre-training and that a relatively small amount of fine-tuning data can substantially reshape model behaviour, motivating the superficial alignment hypothesis.
- **Du et al. (2025)** [8] provide direct mechanistic evidence that post-training affects some behavioural directions, especially refusal, more than core factual knowledge representations.

B Implications for the Present Study

The literature review motivates the methodology in three ways.

- It justifies the use of sparse autoencoders as the main interpretability tool, because superposition makes individual neurons an unreliable unit of analysis.
- It motivates comparing a base model with its instruction-tuned counterpart, because post-training is the stage where alignment-relevant behavioural changes are introduced.

- It frames the core empirical question of the project: whether post-training creates new interpretable features, suppresses existing ones, or primarily changes the activation patterns of a largely preserved feature set.

References

- [1] Anthropic. *Claude Haiku 4.5*. Jan. 1, 2025. URL: <https://www.anthropic.com/claude/haiku>.
- [2] Yuntao Bai et al. *Constitutional AI: Harmlessness from AI Feedback*. Dec. 15, 2022. DOI: 10.48550/arXiv.2212.08073. arXiv: 2212.08073[cs]. URL: <http://arxiv.org/abs/2212.08073> (visited on 2026).
- [3] Yuntao Bai et al. *Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback*. Apr. 12, 2022. DOI: 10.48550/arXiv.2204.05862. arXiv: 2204.05862[cs]. URL: <http://arxiv.org/abs/2204.05862> (visited on 2026).
- [4] Trenton Bricken et al. *Towards Monosemanticity: Decomposing Language Models With Dictionary Learning*. 2023. URL: <https://transformer-circuits.pub/2023/monosemantic-features/>.
- [5] Patrick Chao et al. *JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models*. Mar. 28, 2024. DOI: 10.48550/arXiv.2404.01318. arXiv: 2404.01318[cs.CR]. URL: <http://arxiv.org/abs/2404.01318>.
- [6] Paul Christiano et al. *Deep reinforcement learning from human preferences*. Feb. 17, 2023. DOI: 10.48550/arXiv.1706.03741. arXiv: 1706.03741[stat]. URL: <http://arxiv.org/abs/1706.03741> (visited on 2026).
- [7] Hoagy Cunningham et al. *Sparse Autoencoders Find Highly Interpretable Features in Language Models*. Oct. 4, 2023. DOI: 10.48550/arXiv.2309.08600. arXiv: 2309.08600[cs]. URL: <http://arxiv.org/abs/2309.08600> (visited on 2026).
- [8] Hongzhe Du et al. *How Post-Training Reshapes LLMs: A Mechanistic View on Knowledge, Truthfulness, Refusal, and Confidence*. version: 2. Aug. 11, 2025. DOI: 10.48550/arXiv.2504.02904. arXiv: 2504.02904[cs]. URL: <http://arxiv.org/abs/2504.02904> (visited on 2026).

- [9] Nelson Elhage et al. *Toy Models of Superposition*. Sept. 21, 2022. DOI: 10 . 48550 / arXiv . 2209 . 10652. arXiv: 2209 . 10652[cs]. URL: <http://arxiv.org/abs/2209.10652> (visited on 2026).
- [10] Leo Gao et al. *Scaling and evaluating sparse autoencoders*. June 6, 2024. DOI: 10 . 48550 / arXiv . 2406 . 04093. arXiv: 2406 . 04093[cs]. URL: <http://arxiv.org/abs/2406.04093> (visited on 2026).
- [11] Mor Geva et al. “Transformer Feed-Forward Layers Are Key-Value Memories”. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. EMNLP 2021. Ed. by Marie-Francine Moens et al. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 5484–5495. DOI: 10 . 18653 / v1 / 2021 . emnlp - main . 446. URL: <https://aclanthology.org/2021.emnlp-main.446/> (visited on 2026).
- [12] Evan Hubinger et al. *Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training*. Jan. 17, 2024. DOI: 10 . 48550 / arXiv . 2401 . 05566. arXiv: 2401 . 05566[cs]. URL: <http://arxiv.org/abs/2401.05566> (visited on 2026).
- [13] Samyak Jain et al. *Mechanistically analyzing the effects of fine-tuning on procedurally defined tasks*. Aug. 21, 2024. DOI: 10 . 48550 / arXiv . 2311 . 12786. arXiv: 2311 . 12786[cs]. URL: <http://arxiv.org/abs/2311.12786> (visited on 2026).
- [14] Samyak Jain et al. *What Makes and Breaks Safety Fine-tuning? A Mechanistic Study*. 2024. arXiv: 2407 . 10264 [cs.LG]. URL: <https://arxiv.org/abs/2407.10264>.
- [15] Adam Karvonen et al. *SAEBench: A Comprehensive Benchmark for Sparse Autoencoders in Language Model Interpretability*. June 4, 2025. DOI: 10 . 48550 / arXiv . 2503 . 09532. arXiv: 2503 . 09532[cs]. URL: <http://arxiv.org/abs/2503.09532> (visited on 2026).
- [16] Simon Kornblith et al. *Similarity of Neural Network Representations Revisited*. July 19, 2019. DOI: 10 . 48550 / arXiv . 1905 . 00414. arXiv: 1905 . 00414[cs]. URL: <http://arxiv.org/abs/1905.00414> (visited on 2026).

- [17] Tom Lieberum et al. *Gemma Scope: Open Sparse Autoencoders Everywhere All At Once on Gemma 2*. Aug. 19, 2024. DOI: 10.48550/arXiv.2408.05147. arXiv: 2408.05147[cs]. URL: <http://arxiv.org/abs/2408.05147> (visited on 2026).
- [18] Stephanie Lin, Jacob Hilton, and Owain Evans. *TruthfulQA: Measuring How Models Mimic Human Falsehoods*. 2022. DOI: 10.48550/arXiv.2109.07958. arXiv: 2109.07958[cs]. URL: <http://arxiv.org/abs/2109.07958>.
- [19] Kevin Meng et al. *Locating and Editing Factual Associations in GPT*. Jan. 13, 2023. DOI: 10.48550/arXiv.2202.05262. arXiv: 2202.05262[cs]. URL: <http://arxiv.org/abs/2202.05262> (visited on 2026).
- [20] Tomas Mikolov et al. *Efficient Estimation of Word Representations in Vector Space*. Sept. 7, 2013. DOI: 10.48550/arXiv.1301.3781. arXiv: 1301.3781[cs]. URL: <http://arxiv.org/abs/1301.3781> (visited on 2026).
- [21] Chris Olah, Alexander Mordvintsev, and Ludwig Schubert. “Feature Visualization”. In: *Distill* 2.11 (Nov. 7, 2017), 10.23915/distill.00007. ISSN: 2476-0757. DOI: 10.23915/distill.00007. URL: <https://distill.pub/2017/feature-visualization> (visited on 2026).
- [22] Chris Olah et al. “Zoom In: An Introduction to Circuits”. In: *Distill* 5.3 (Mar. 10, 2020), 10.23915/distill.00024.001. ISSN: 2476-0757. DOI: 10.23915/distill.00024.001. URL: <https://distill.pub/2020/circuits/zoom-in> (visited on 2026).
- [23] Long Ouyang et al. *Training language models to follow instructions with human feedback*. Mar. 4, 2022. DOI: 10.48550/arXiv.2203.02155. arXiv: 2203.02155[cs]. URL: <http://arxiv.org/abs/2203.02155> (visited on 2026).
- [24] Kiho Park, Yo Joong Choe, and Victor Veitch. *The Linear Representation Hypothesis and the Geometry of Large Language Models*. July 17, 2024. DOI: 10.48550/arXiv.2311.03658. arXiv: 2311.03658[cs]. URL: <http://arxiv.org/abs/2311.03658> (visited on 2026).

- [25] Guilherme Penedo et al. *The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale*. 2024. DOI: 10.48550/arXiv.2406.17557. arXiv: 2406.17557[cs]. URL: <http://arxiv.org/abs/2406.17557>.
- [26] Kenny Peng et al. *Use Sparse Autoencoders to Discover Unknown Concepts, Not to Act on Known Concepts*. June 30, 2025. DOI: 10.48550/arXiv.2506.23845. arXiv: 2506.23845[cs]. URL: <http://arxiv.org/abs/2506.23845> (visited on 2026).
- [27] Nikhil Prakash et al. *Fine-Tuning Enhances Existing Mechanisms: A Case Study on Entity Tracking*. Feb. 22, 2024. DOI: 10.48550/arXiv.2402.14811. arXiv: 2402.14811[cs]. URL: <http://arxiv.org/abs/2402.14811> (visited on 2026).
- [28] Alec Radford, Rafal Jozefowicz, and Ilya Sutskever. *Learning to Generate Reviews and Discovering Sentiment*. Apr. 6, 2017. DOI: 10.48550/arXiv.1704.01444. arXiv: 1704.01444[cs]. URL: <http://arxiv.org/abs/1704.01444> (visited on 2026).
- [29] Rafael Rafailov et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. July 29, 2024. DOI: 10.48550/arXiv.2305.18290. arXiv: 2305.18290[cs]. URL: <http://arxiv.org/abs/2305.18290> (visited on 2026).
- [30] Senthooran Rajamanoharan et al. *Improving Dictionary Learning with Gated Sparse Autoencoders*. Apr. 30, 2024. DOI: 10.48550/arXiv.2404.16014. arXiv: 2404.16014[cs]. URL: <http://arxiv.org/abs/2404.16014> (visited on 2026).
- [31] Adam Scherlis et al. *Polysemanticity and Capacity in Neural Networks*. Mar. 25, 2025. DOI: 10.48550/arXiv.2210.01892. arXiv: 2210.01892[cs]. URL: <http://arxiv.org/abs/2210.01892> (visited on 2026).
- [32] Rohan Taori et al. *Stanford Alpaca*. Instruction-following dataset repository. 2023. URL: https://github.com/tatsu-lab/stanford_alpaca.
- [33] Adly Templeton et al. *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. 2024. URL: <https://transformer-circuits.pub/2024/scaling-monosemanticity/>.

- [34] Kevin Wang et al. *Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 small*. Nov. 1, 2022. DOI: 10.48550/arXiv.2211.00593. arXiv: 2211.00593[cs]. URL: <http://arxiv.org/abs/2211.00593> (visited on 2026).
- [35] Jason Wei et al. *Finetuned Language Models Are Zero-Shot Learners*. Feb. 8, 2022. DOI: 10.48550/arXiv.2109.01652. arXiv: 2109.01652[cs]. URL: <http://arxiv.org/abs/2109.01652> (visited on 2026).
- [36] B. L. Welch. “The Generalization of “Student’s” Problem When Several Different Population Variances Are Involved”. In: *Biometrika* 34.1-2 (Jan. 1, 1947), pp. 28–35. DOI: 10.1093/biomet/34.1-2.28. URL: <https://doi.org/10.1093/biomet/34.1-2.28>.
- [37] Yang Xu et al. *Tracking the Feature Dynamics in LLM Training: A Mechanistic Study*. 2025. arXiv: 2412.17626 [cs.LG]. URL: <https://arxiv.org/abs/2412.17626> (visited on 2026).
- [38] Chunting Zhou et al. *LIMA: Less Is More for Alignment*. May 18, 2023. DOI: 10.48550/arXiv.2305.11206. arXiv: 2305.11206[cs]. URL: <http://arxiv.org/abs/2305.11206> (visited on 2026).
- [39] Daniel M. Ziegler et al. *Fine-Tuning Language Models from Human Preferences*. Jan. 8, 2020. DOI: 10.48550/arXiv.1909.08593. arXiv: 1909.08593[cs]. URL: <http://arxiv.org/abs/1909.08593> (visited on 2026).
- [40] Andy Zou et al. *Universal and Transferable Adversarial Attacks on Aligned Language Models*. Dec. 20, 2023. DOI: 10.48550/arXiv.2307.15043. arXiv: 2307.15043[cs]. URL: <http://arxiv.org/abs/2307.15043> (visited on 2026).
- [41] Andy Zou et al. *Representation Engineering: A Top-Down Approach to AI Transparency*. Mar. 3, 2025. DOI: 10.48550/arXiv.2310.01405. arXiv: 2310.01405[cs]. URL: <http://arxiv.org/abs/2310.01405> (visited on 2026).